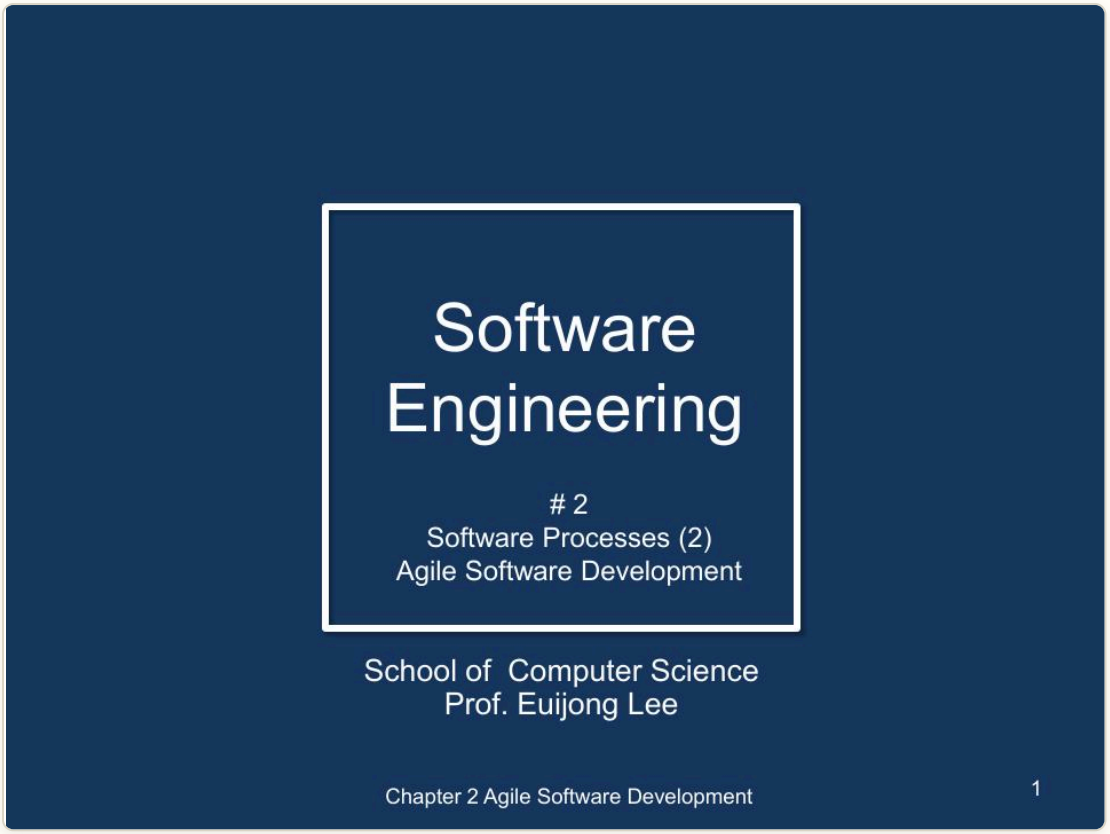
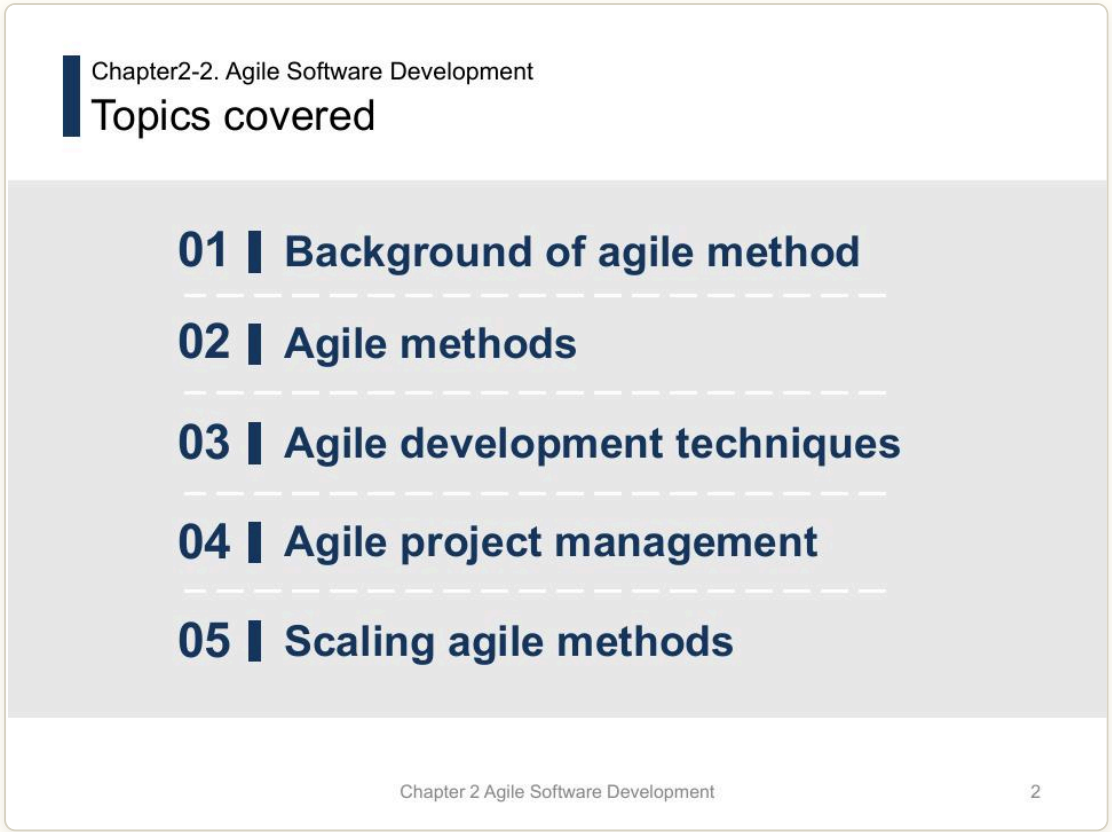


소프트웨어공학 #2 — 소프트웨어 프로세스 (2): 애자일 SW 개발

- 이의종 교수님

필기 ✍️

💡 이해 흐름: 배경 → 애자일 선언·원칙 → XP 기법 → 스크럼 → 대규모 적용



다루는 내용

- 애자일 방법의 배경 / 애자일 방법 / 애자일 개발 기법(XP) / 애자일 프로젝트 관리(스크럼) / 애자일 규모 확장

필기 ✍️

— 넘어가도 되는 쪽

Chapter2-2. Agile Software Development

Agile?

- Agile success cases
  - https://hrbulletin.net/casestudy/%EB%A0%88%EA%B3%A0%EC%9D%98-%EC%95%A0%EC%9E%90%EC%9D%BC-%EC%A0%84%ED%99%98-1%EB%85%84%EC%9D%98-%EC%84%B1%EA%B3%BC/
  - https://news.skhynix.co.kr/post/Meeting-of-SK-hynix-and-Agile

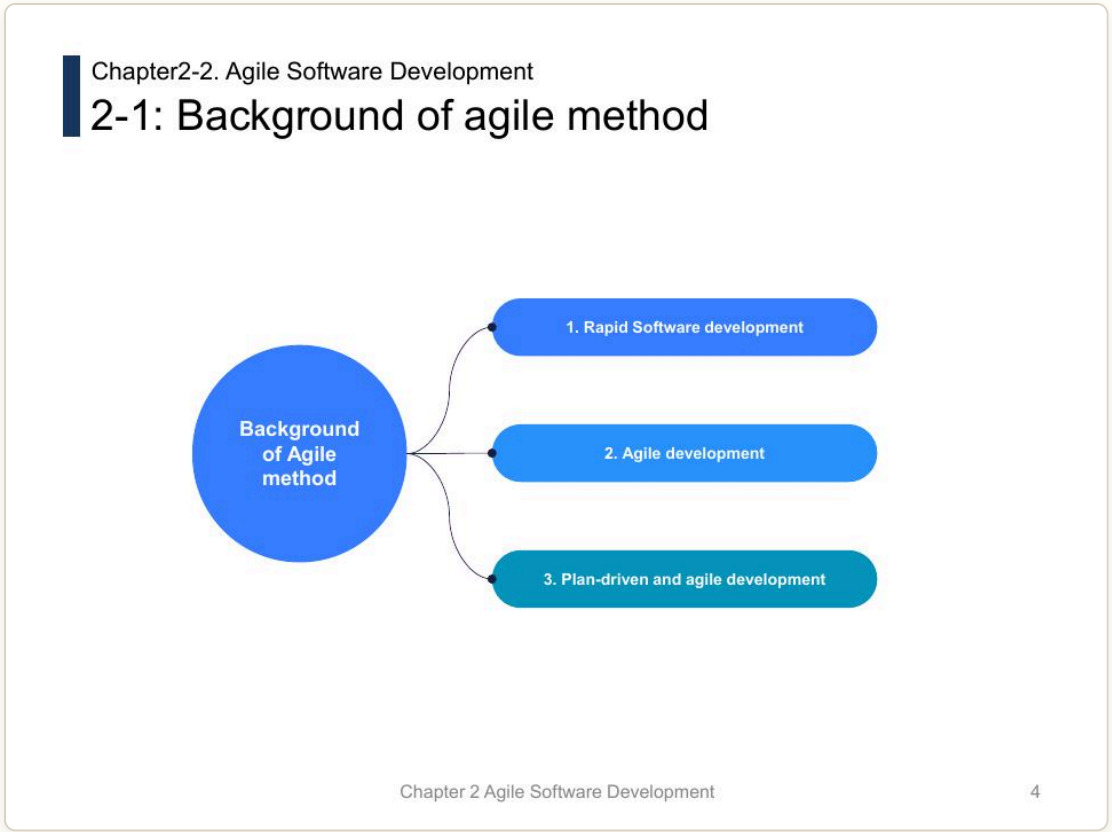
Chapter 2 Agile Software Development 3

애자일?

- 애자일 성공 사례 기사 링크 (레고의 애자일 전환, SK하이닉스와 애자일)

필기 ✍️

— 넘어가도 되는 쪽



2-1: 애자일 방법의 배경

- 1. 신속한 SW 개발
- 2. 애자일 개발
- 3. 계획 주도 vs 애자일 개발

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Background of agile method

Background of agile method

[Rapid software development : 신속한 소프트웨어 개발 **]**

- **Rapid development and delivery is now often the most important requirement for software systems**
  - Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements
  - Software has to evolve quickly to reflect changing business needs.
- **Plan-driven development**
  - Essential for some types of system but does not meet these business needs.
- **Agile development methods**
  - Emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems

Chapter 2 Agile Software Development 5

애자일의 배경: 신속한 SW 개발

- **빠른 개발과 전달**이 이제 SW 시스템의 **가장 중요한 요구**인 경우가 많다
  - 기업 환경이 빠르게 바뀌어 **안정된 요구사항을 만드는 것이 사실상 불가능**
  - SW는 바뀌는 비즈니스 요구를 반영해 빠르게 진화해야
- **계획 주도 개발** — 어떤 종류의 시스템엔 필수지만 이런 비즈니스 요구는 못 맞춤
- **애자일 개발 방법** — **1990년대 후반** 등장, 동작하는 SW의 전달 시간을 획기적으로 줄이는 것이 목표

필기 ✍️

★ 시험 등장 시기: late 1990s

Background of agile method

Agile development

- Program specification, design and implementation are **inter-leaved**
- The system is developed **as a series of versions or increments** with stakeholders involved in version specification and evaluation
- **Frequent delivery** of new versions for evaluation
- **Extensive tool support** (e.g. automated testing tools) used to support development.
- **Minimal documentation** – focus on working code

Chapter 2 Agile Software Development 6

애자일 개발

- 명세·설계·구현이 **번갈아(inter-leaved)** 진행
- 시스템을 **버전(증분)의 연속**으로 개발 — 이해관계자가 버전 명세와 평가에 참여
- 평가를 위해 새 버전을 **자주** 전달
- **도구 지원**을 많이 씀 (예: 자동 테스트 도구)
- **문서는 최소** — 동작하는 코드에 집중

필기 ✍️

★ 시험 애자일 특징 5개: 번갈아 진행 · 증분 · 잦은 전달 · 도구 지원 · 최소 문서

Background of agile method

Plan-driven and Agile development (1)

- **Plan-driven** development
  - **A plan-driven approach** to software engineering is based around **separate development stages** with the **outputs** to be produced at each of these stages planned in advance.
  - Not necessarily waterfall model – plan-driven, incremental development is possible
  - Iteration occurs within activities.

Chapter 2 Agile Software Development 7

계획 주도 vs 애자일 ① 계획 주도

- 계획 주도 접근은 **분리된 개발 단계**를 기반으로, 각 단계의 산출물을 **미리 계획**
- **꼭 폭포수일 필요는 없다** — 계획 주도의 점증적 개발도 가능
- 반복은 **활동 안에서** 일어난다

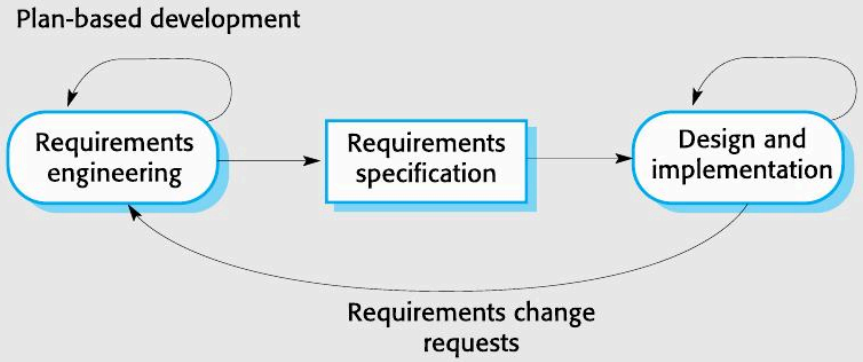
필기 ✍️

⚠ 주의 '계획 주도 = 폭포수' ⟶ X (계획 주도 점증적 개발도 있음)

Background of agile method

Plan-driven and Agile development (1)

- **Plan-driven** development



Chapter 2 Agile Software Development 8

계획 주도 개발 (그림)

- 요구공학 → 요구 명세 → 설계·구현
  - 요구공학·설계 단계 안에서 각자 반복(제자리 화살표)
  - 설계·구현 중 생긴 **요구 변경 요청**은 요구공학으로 되돌아감

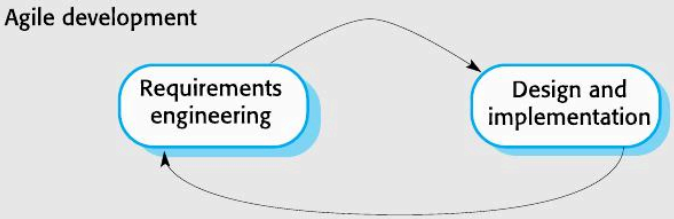
필기 ✍️

💡 이해 핵심: 중간에 '요구 명세'라는 문서가 산출물로 딱 나온다

Background of agile method

Plan-driven and Agile development (2)

- **Agile** development
  - Specification, design, implementation and testing are **inter-leaved**
  - The **outputs** from the development process are decided through a **process of negotiation** during the software development process.

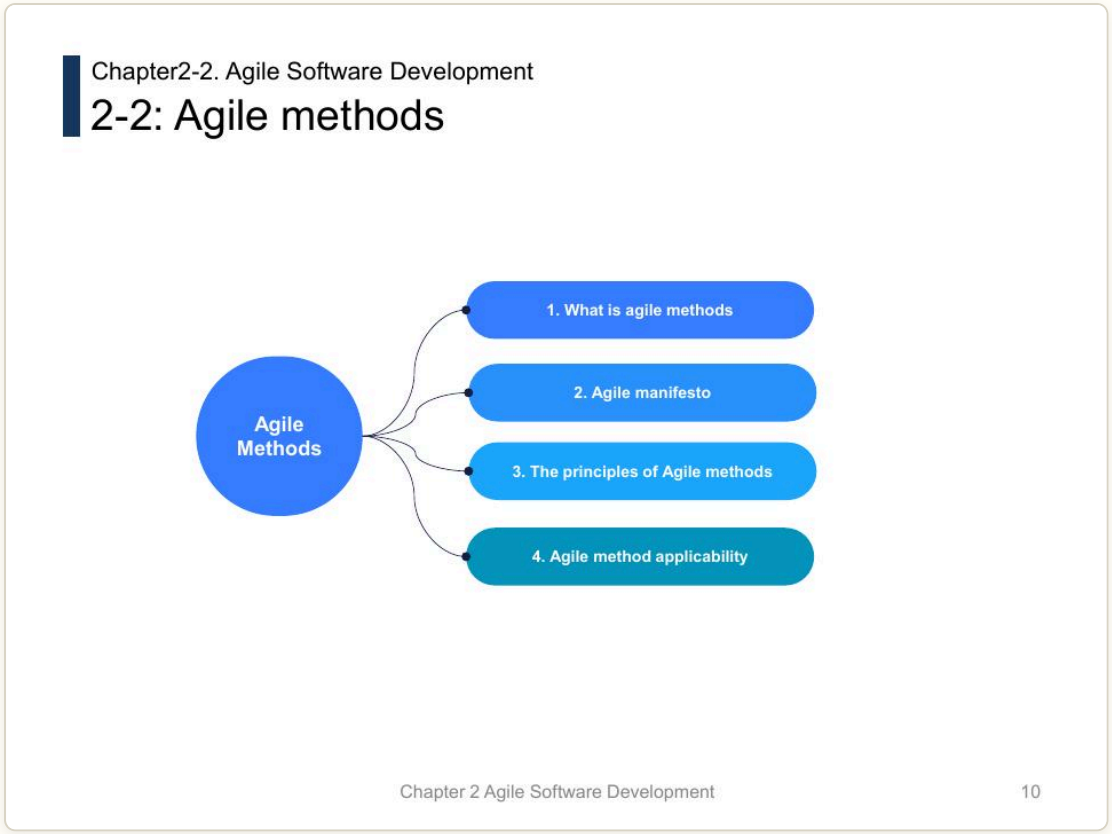


계획 주도 vs 애자일 ② 애자일

- 명세·설계·구현·테스트가 **번갈아** 진행
- 개발 산출물은 개발 도중 **협상**을 통해 정해진다

필기 ✍️

★ 시험 계획 주도: 산출물을 미리 계획 / 애자일: 산출물을 협상으로 결정



2-2: 애자일 방법

- 1. 애자일 방법이란
- 2. 애자일 선언
- 3. 애자일 방법의 원칙
- 4. 애자일 방법의 적용 분야

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Agile methods

What is Agile methods

[What is agile methods?]

Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods.

- Characteristics of agile methods
  - **Focus on the code** rather than the design
  - Are based on an **iterative approach** to software development
  - Are intended to deliver working software **quickly** and **evolve** this quickly to meet changing requirements.

- The aim of agile methods
  - **reduce overheads** in the software process (e.g. by limiting documentation)
  - able to respond **quickly to changing** requirements without excessive rework.

Chapter 2 Agile Software Development 11

애자일 방법이란?

- 1980~90년대 SW 설계 방법의 **오버헤드에 대한 불만**에서 애자일이 생겼다
- **특징**
  - 설계보다 **코드**에 집중
  - **반복적(iterative)** 개발 접근
  - 동작하는 SW를 **빨리** 전달하고 바뀌는 요구에 맞춰 빨리 진화
- **목표**
  - 프로세스 **오버헤드 감소** (예: 문서 제한)
  - 과도한 재작업 없이 요구 변경에 **빠르게 대응**

필기 ✍️

★ 시험 목표 2개: 오버헤드↓ · 변경에 빠른 대응



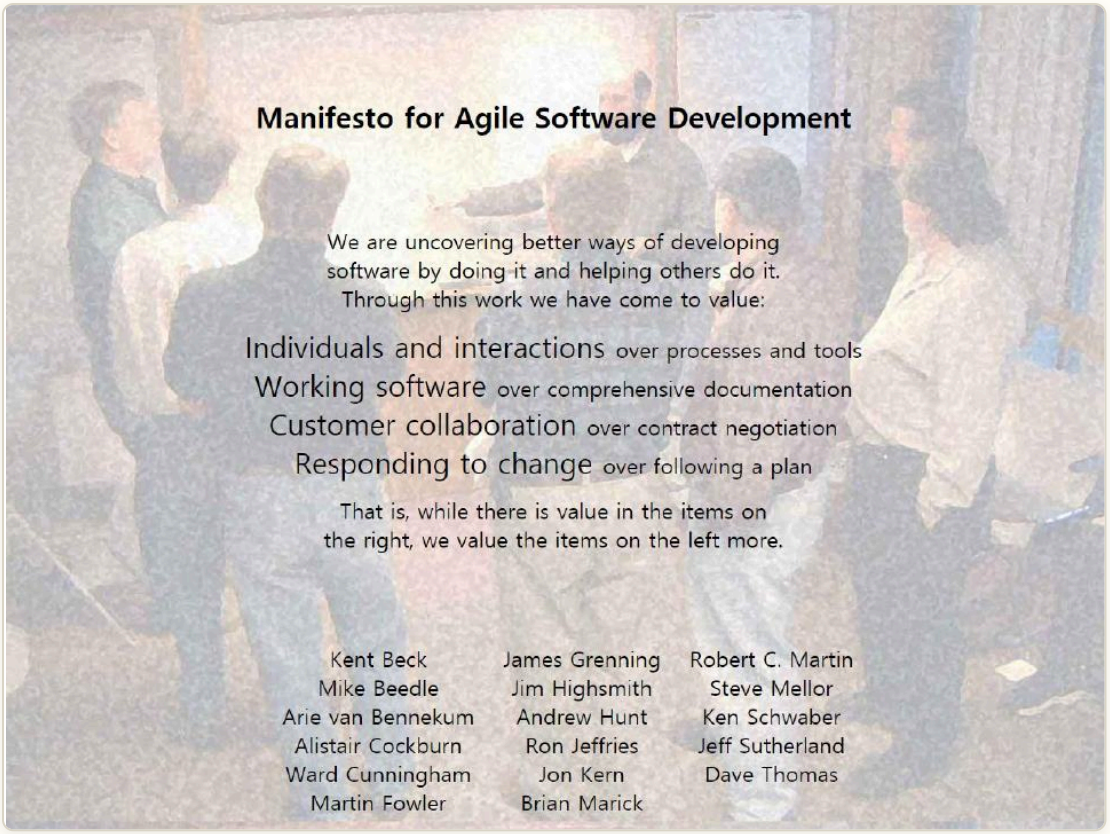
애자일 선언 (Agile Manifesto)

- 우리는 SW를 직접 개발하고 남을 도우며 더 나은 개발 방법을 찾고 있다. 그 결과 다음을 가치 있게 여긴다:
  - **공정과 도구**보다 **개인과 상호작용**
  - **포괄적인 문서**보다 **동작하는 SW**
  - **계약 협상**보다 **고객과의 협력**
  - **계획을 따르기**보다 **변화에 대응하기**
- 오른쪽 것도 가치가 있지만, **왼쪽 것을 더** 가치 있게 여긴다

필기 ✍️

★ 시험 선언 4줄 — 'A over B'에서 A가 더 중요. 짝 맞추기 문제 단골!

⚠ 주의 오른쪽(문서·계약·계획)이 '필요 없다'는 뜻이 아님

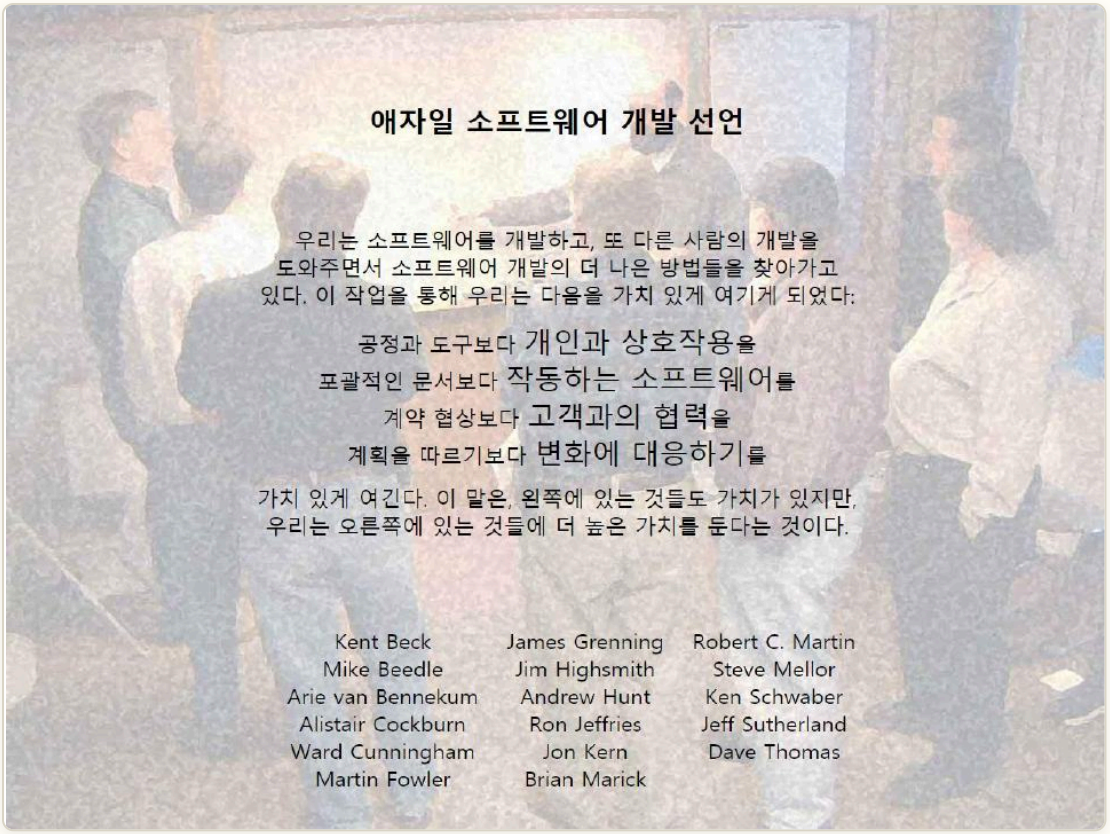


애자일 선언 원문 (사진)

- Manifesto for Agile Software Development — 4개 가치 + 서명자 17명 (Kent Beck, Martin Fowler, Ken Schwaber, Jeff Sutherland 등)

필기 ✍️

— 넘어가도 되는 쪽

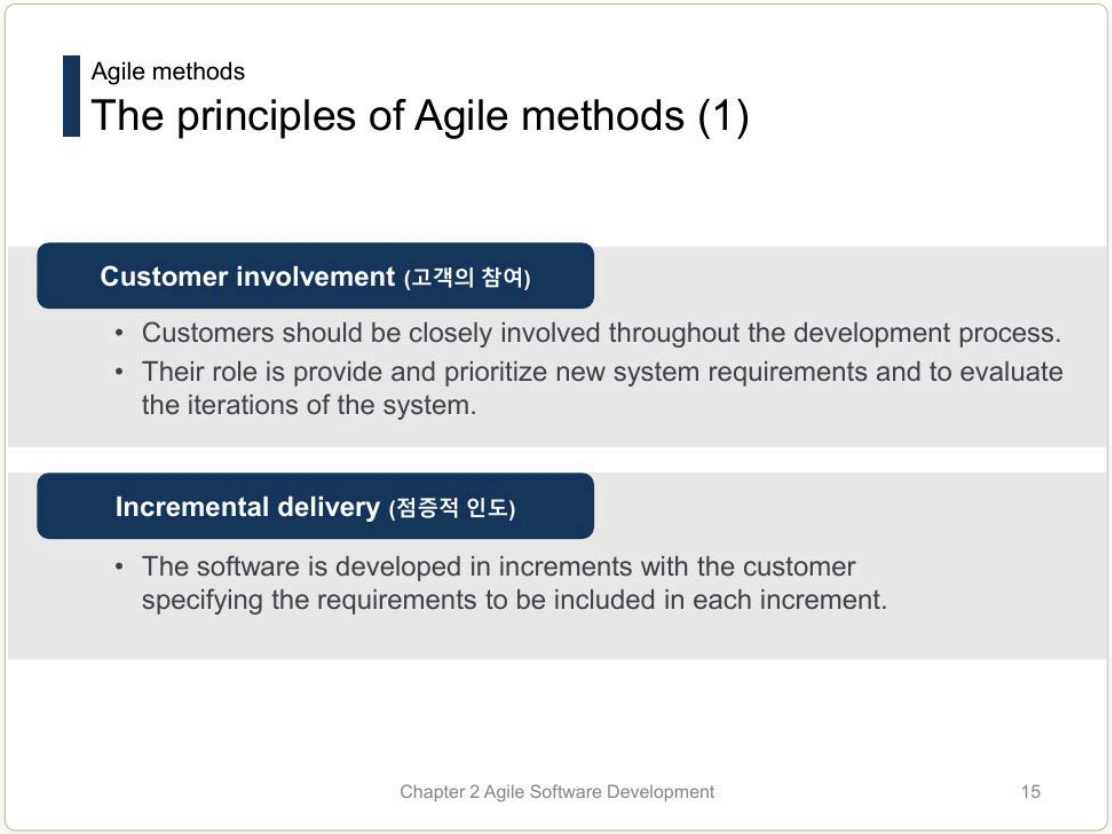


애자일 소프트웨어 개발 선언 (한국어판)

- 공정과 도구보다 **개인과 상호작용**을
- 포괄적인 문서보다 **작동하는 소프트웨어**를
- 계약 협상보다 **고객과의 협력**을
- 계획을 따르기보다 **변화에 대응하기**를 가치 있게 여긴다

필기 ✍️

— 넘어가도 되는 쪽



애자일 원칙 ① 고객 참여 · 점증적 인도

- **Customer involvement (고객의 참여)**
  - 고객은 개발 내내 **밀접하게** 참여
  - 역할: 새 요구를 제시·우선순위 지정, 반복 결과물 평가
- **Incremental delivery (점증적 인도)**
  - SW를 증분으로 개발하고, 각 증분에 넣을 요구를 **고객이 정한다**

필기 ✍️

— 넘어가도 되는 쪽

Agile methods

The principles of Agile methods (2)

Embrace change (변화의 수용)

- Expect the system requirements to change and so design the system to accommodate these changes.

Maintain simplicity (단순성 유지)

- Focus on simplicity in both the software being developed and in the development process.
- Wherever possible, actively work to eliminate complexity from the system.

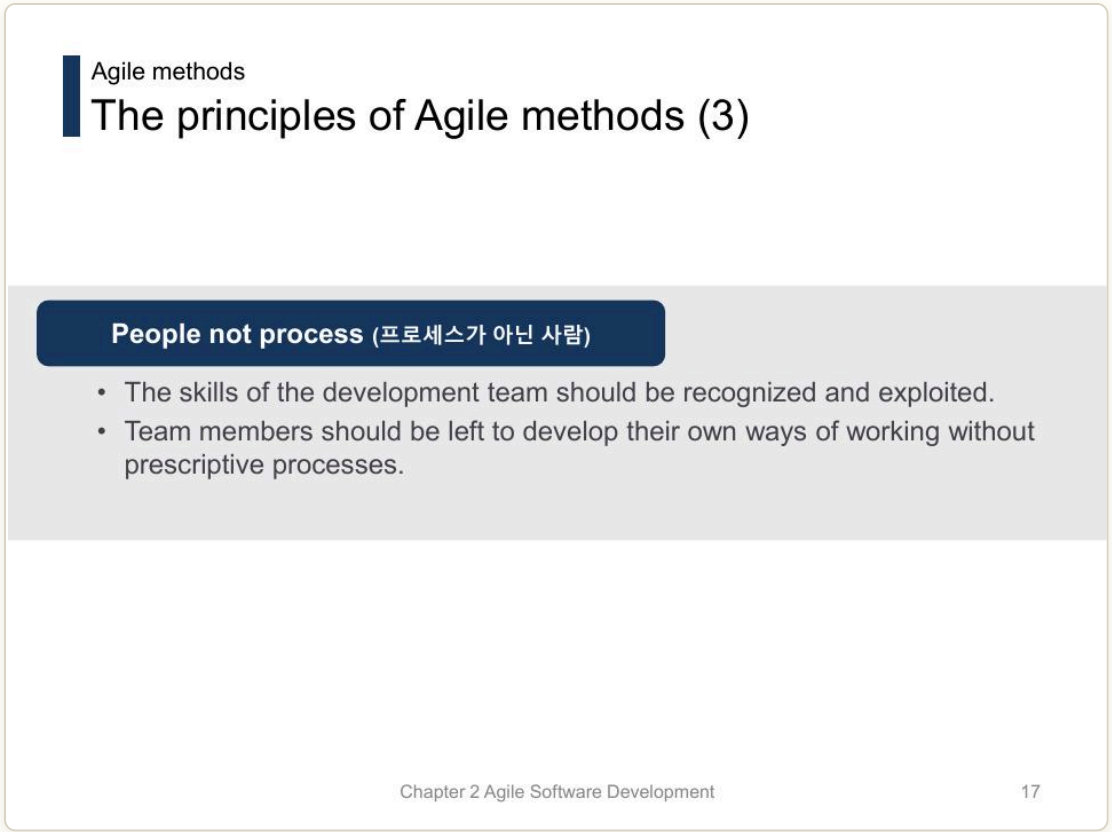
Chapter 2 Agile Software Development 16

애자일 원칙 ② 변화 수용 · 단순성 유지

- **Embrace change (변화의 수용)**
  - 요구가 **바뀔 것을 예상**하고 그 변화를 받아들이도록 설계
- **Maintain simplicity (단순성 유지)**
  - 개발하는 SW와 개발 프로세스 **둘 다** 단순하게
  - 가능하면 시스템의 복잡성을 적극적으로 없앤다

필기 ✍️

— 넘어가도 되는 쪽



애자일 원칙 ③ 프로세스가 아닌 사람

- **People not process**
  - 개발팀의 **기술(역량)**을 인정하고 활용
  - 팀원이 **정해진(prescriptive) 프로세스 없이** 자기 방식으로 일하게 둔다

필기 ✍️

★ 시험 원칙 5개: 고객 참여 · 점증적 인도 · 변화 수용 · 단순성 유지 · 사람(프로세스 X)

Agile methods

Agile method applicability : 적용가능 분야

- **Product development**
  - a software company is developing a **small or medium-sized** product for sale.
  - Virtually all software products and apps are now developed using an agile approach
- **Custom system development within an organization**
  - there is a **clear commitment** from the **customer to become involved** in the development process
  - there are few external rules and regulations that affect the software.

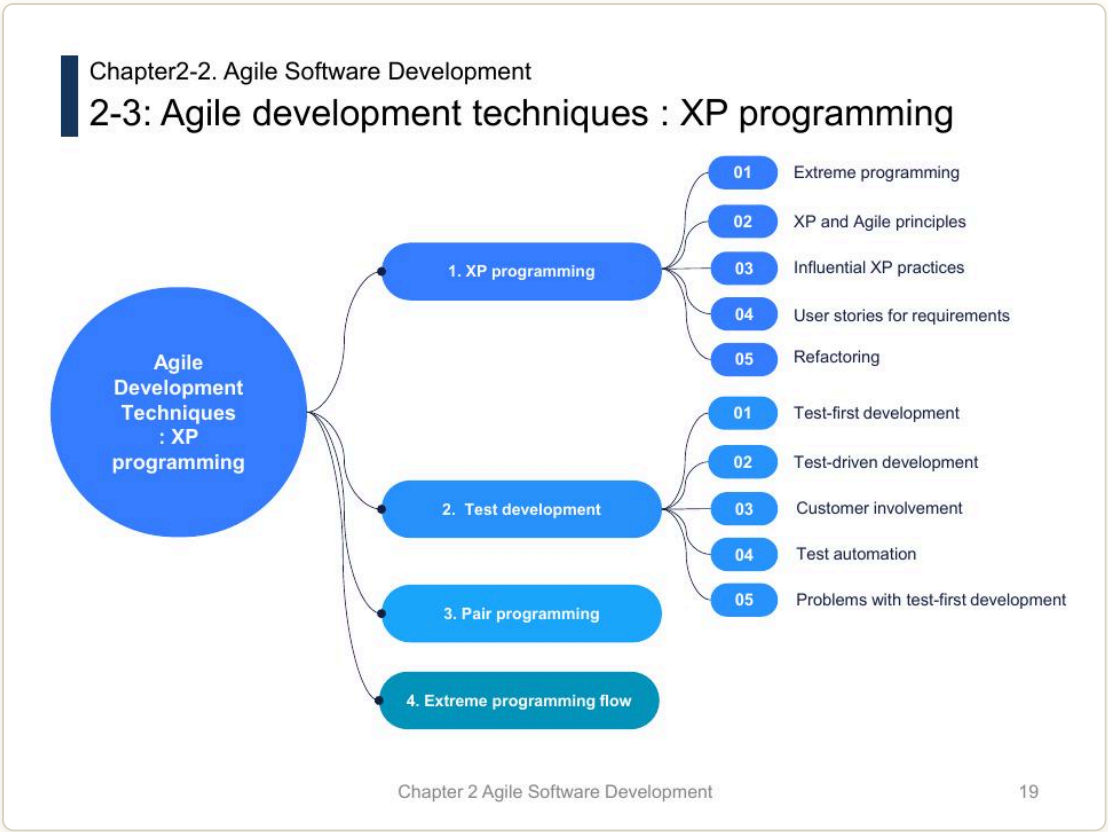
Chapter 2 Agile Software Development 18

애자일 적용 분야

- **Product development (제품 개발)**
  - SW 회사가 판매용 **중소 규모** 제품을 개발
  - 사실상 거의 모든 SW 제품과 앱이 이제 애자일로 개발된다
- **조직 내 맞춤형 시스템 개발**
  - 고객이 개발에 **참여하겠다는 분명한 약속**이 있을 때
  - SW에 영향을 주는 **외부 규칙·규제가 적을** 때

필기 ✍️

★ 시험 맞는 곳: 중소 제품 / 고객 참여 확실 + 규제 적은 사내 시스템



2-3: 애자일 개발 기법 — XP

- 1. XP — XP와 애자일 원칙, 주요 실천, 요구사항용 사용자 스토리, 리팩토링
- 2. 테스트 개발 — 테스트 우선, 테스트 주도, 고객 참여, 자동화, 테스트 우선의 문제
- 3. 짝 프로그래밍
- 4. XP 흐름

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Agile development techniques : XP programming

XP programming

[Extreme programming]

- A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- Extreme Programming (XP) takes an '**extreme** : 극단적' approach to iterative development.
  - **New versions** may be built **several times per day**;
  - **Increments** are delivered to customers **every 2 weeks**;
  - All **tests must be run for every build** and the build is only accepted if tests run successfully.

Chapter 2 Agile Software Development 20

XP (Extreme Programming)

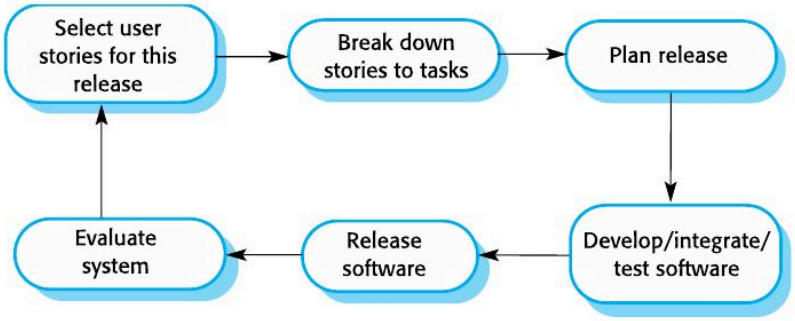
- **1990년대 후반** 개발된 아주 영향력 큰 애자일 방법 — 다양한 애자일 기법을 도입
- 반복 개발에 **'극단적(extreme)'** 접근
  - 새 버전을 **하루에도 여러 번** 빌드
  - 증분을 **2주마다** 고객에게 전달
  - 빌드마다 **모든 테스트**를 돌리고, 통과해야만 빌드 인정

필기 ✍️

★ 시험 XP 숫자: 하루 여러 번 빌드 · 2주마다 전달 · 모든 테스트 통과

Agile development techniques : XP programming

XP programming



Chapter 2 Agile Software Development 21

XP 릴리스 주기 (그림)

- 이번 릴리스 스토리 선택 → 스토리를 태스크로 쪼갬 → 릴리스 계획 → SW 개발·통합·테스트 → SW 릴리스 → 시스템 평가 → (다시 스토리 선택)

필기 ✍️

★ 시험 XP 사이클 6단계 순서 — 순서 맞추기 대비

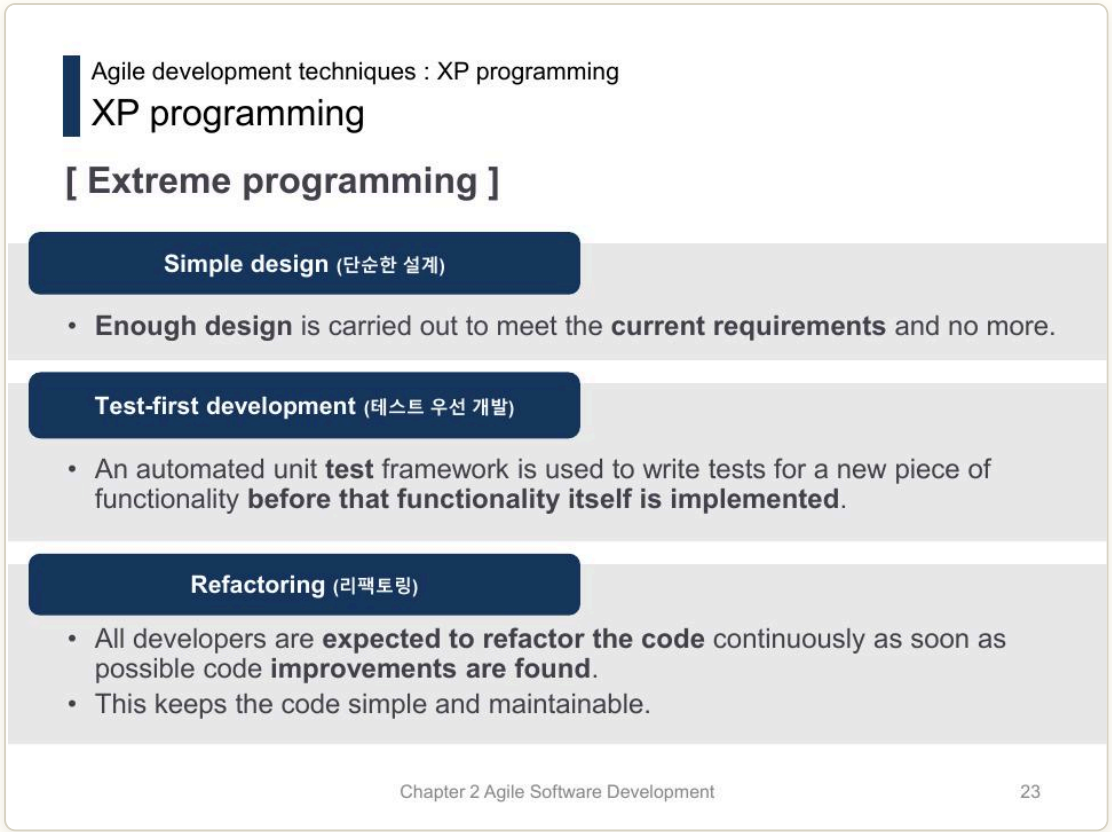


XP 실천 ① 점진적 계획 · 소규모 릴리스

- **Incremental planning (점진적인 계획)**
  - 요구를 **스토리 카드**에 기록, 릴리스에 넣을 스토리는 **가용 시간과 상대적 우선순위**로 결정
  - 개발자가 스토리를 개발 **태스크**로 쪼갬
- **Small releases (소규모 릴리스)**
  - 비즈니스 가치를 주는 **최소한의 유용한 기능**을 먼저 개발
  - 릴리스는 **자주**, 첫 릴리스에 기능을 점진적으로 추가

필기 ✍️

— 넘어가도 되는 쪽

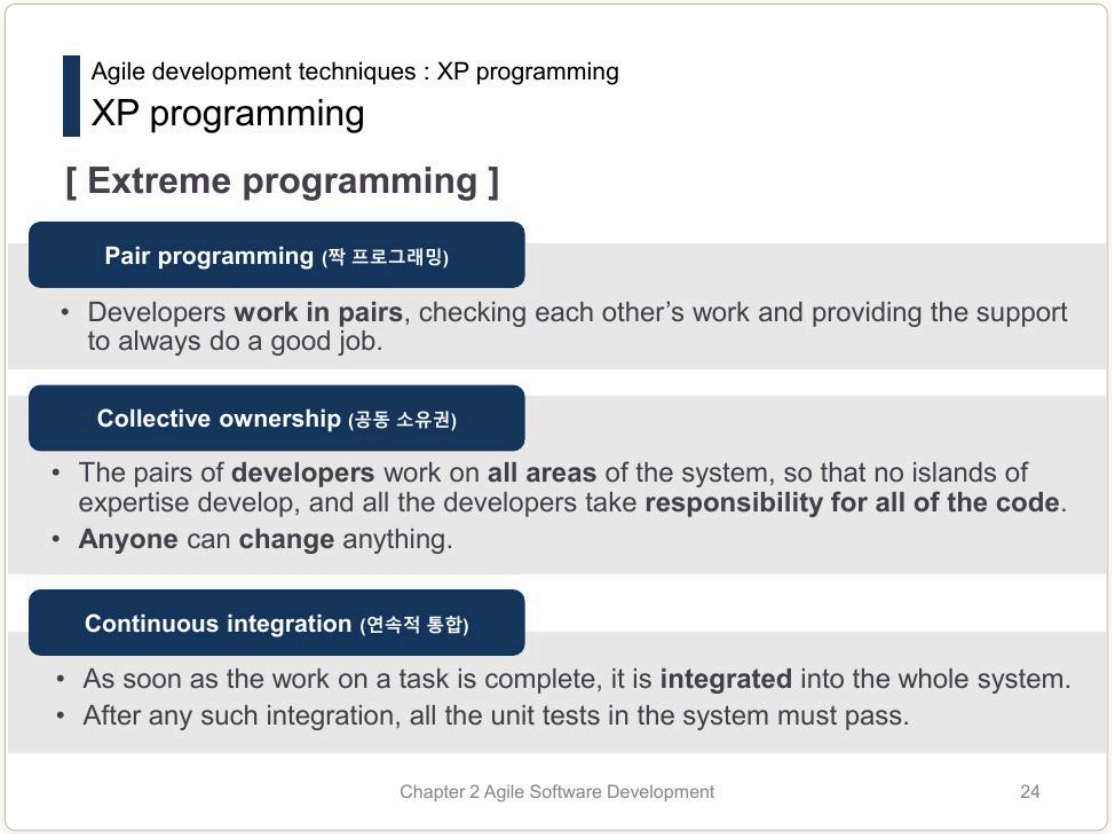


XP 실천 ② 단순한 설계 · 테스트 우선 · 리팩토링

- **Simple design (단순한 설계)** — 현재 요구를 만족할 만큼**만** 설계, 그 이상은 X
- **Test-first development (테스트 우선 개발)** — 자동 단위 테스트 프레임워크로 기능을 **구현하기 전에** 테스트를 작성
- **Refactoring (리팩토링)** — 개선점이 보이면 **바로, 계속** 코드를 리팩토링 → 코드를 단순하고 유지보수 쉽게

필기 ✍️

— 넘어가도 되는 쪽

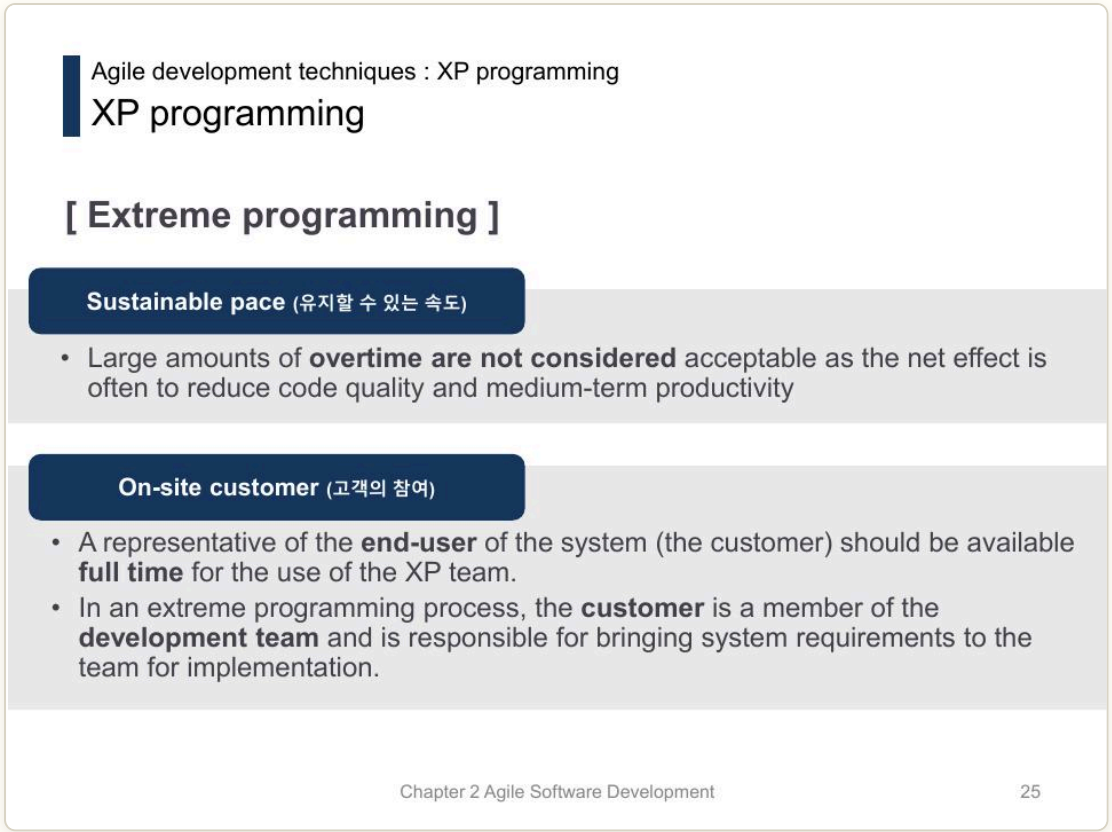


XP 실천 ③ 짝 프로그래밍 · 공동 소유권 · 연속적 통합

- **Pair programming (짝 프로그래밍)** — 둘씩 일하며 서로 작업을 검토·지원
- **Collective ownership (공동 소유권)** — 짝들이 시스템 **모든 영역**에서 일해 전문 지식의 '섬'이 안 생김, 모두가 모든 코드에 책임
  - **누구나 무엇이든 바꿀 수 있다**
- **Continuous integration (연속적 통합)** — 태스크가 끝나면 **즉시** 전체 시스템에 통합
  - 통합 후엔 시스템의 **모든 단위 테스트가 통과**해야

필기 ✍️

— 넘어가도 되는 쪽



XP 실천 ④ 유지할 수 있는 속도 · 현장 고객

- **Sustainable pace (유지할 수 있는 속도)** — **과도한 야근은 허용 안 함**: 결국 코드 품질과 중기 생산성을 떨어뜨림
- **On-site customer (현장 고객)** — 최종 사용자 대표(고객)가 XP 팀을 위해 **풀타임**으로 있어야
  - XP에서 고객은 **개발팀의 일원**, 구현할 요구를 팀에 가져올 책임

필기 ✍️

★ 시험 XP 실천 10개: 점진적 계획 · 소규모 릴리스 · 단순 설계 · 테스트 우선 · 리팩토링 · 짝 프로그래밍 · 공동 소유권 · 연속 통합 · 유지 가능한 속도 · 현장 고객

Agile development techniques : XP programming

XP programming

[XP and agile principles]

- **Incremental development** is supported through small, frequent system releases.
- **Customer involvement** means full-time customer engagement with the team.
- People not process through **pair programming**, **collective ownership** and a process that **avoids long working hours.**
- **Change supported** through regular system **releases**.
- Maintaining **simplicity** through constant **refactoring** of code.

Chapter 2 Agile Software Development 26

XP와 애자일 원칙의 대응

- **점증적 개발** ← 작고 잦은 시스템 릴리스
- **고객 참여** ← 팀에 풀타임 고객 참여
- **프로세스가 아닌 사람** ← 짝 프로그래밍, 공동 소유권, 긴 근무를 피하는 프로세스
- **변화 지원** ← 정기적 시스템 릴리스
- **단순성 유지** ← 끊임없는 리팩토링

필기 ✍️

★ 시험 원칙 ↔ XP 실천 짝 — 서술형 대비

Agile development techniques : XP programming

XP programming

Influential XP practices

- Extreme programming has a technical focus and is not easy to integrate with management practice in most organizations.
- Consequently, while agile development uses practices from XP, the method as originally defined is **not widely used**.
- Key practices
  - User stories for specification
  - Refactoring
  - Test-first development
  - Pair programming

Chapter 2 Agile Software Development 27

영향력 있는 XP 실천

- XP는 **기술 중심**이라 대부분 조직의 관리 관행과 통합하기 어렵다
- 그래서 애자일 개발은 XP의 실천을 **가져다 쓰지만**, 원래 정의된 XP 자체는 **널리 쓰이지 않는다**
- 핵심 실천 4개
  - 요구 명세용 **사용자 스토리**
  - **리팩토링**
  - **테스트 우선 개발**
  - **짝 프로그래밍**

필기 ✍️

★ 시험 살아남은 XP 실천 4개: 사용자 스토리 · 리팩토링 · 테스트 우선 · 짝 프로그래밍

Agile development techniques : XP programming

XP programming

User stories for requirements

- In XP, a **customer** or user is part of the **XP team** and is responsible for making **decisions on requirements**.
- User **requirements** are expressed as **user stories or scenarios**.
- These are written on cards and the development team **break them down** into implementation **tasks**.
- These **tasks** are the **basis** of **schedule** and **cost** estimates.
- The customer **chooses** the **stories** for inclusion in the **next release** based on their **priorities** and the **schedule** estimates.

Chapter 2 Agile Software Development 28

요구사항용 사용자 스토리

- XP에서 고객·사용자는 XP 팀의 일원이고 **요구 결정**을 책임진다
- 사용자 요구는 **사용자 스토리나 시나리오**로 표현
- 카드에 적고, 개발팀이 이를 **구현 태스크**로 쪼갬
- 태스크가 **일정·비용 추정**의 기준
- 고객이 **우선순위와 일정 추정**을 보고 다음 릴리스에 넣을 스토리를 고른다

필기 ✍️

★ 시험 스토리 ⟶ 태스크 ⟶ 일정·비용 추정 / 선택은 고객이

Agile development techniques : XP programming

XP programming - A ‘prescribing medication’ story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either ‘current medication’, ‘new medication’ or ‘formulary’.

If you select ‘current medication’, you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, ‘new medication’, the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose ‘formulary’, you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click ‘OK’ or ‘Change’. If you click ‘OK’, your prescription will be recorded on the audit database. If you click ‘Change’, you reenter the ‘Prescribing medication’ process.

Chapter 2 Agile Software Development 29

예: '약 처방' 스토리 카드

- 환자 기록을 열고 약 항목에서 '현재 약 / 새 약 / 처방집(formulary)' 중 선택
- 현재 약 → 용량 확인, 바꾸려면 새 용량 입력 후 처방 확인
- 새 약 → 약 이름 앞글자 입력 → 목록에서 선택 → 확인 → 용량 입력
- 처방집 → 승인된 처방집 검색 → 선택 → 확인 → 용량 입력
- 모든 경우 용량이 **승인 범위 안인지** 검사, 벗어나면 변경 요청
- 확인 후 'OK'면 감사 DB에 기록, 'Change'면 처방 과정 다시

필기 ✍️

— 넘어가도 되는 쪽

Agile development techniques : XP programming

XP programming

[Examples of task cards for prescribing medication]

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.
Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.
Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low.
If within the range, enable the 'Confirm' button.

Chapter 2 Agile Software Development 30

예: 약 처방의 태스크 카드

- Task 1: 처방된 약의 용량 변경
- Task 2: 처방집 선택
- **Task 3: 용량 검사** — 의사가 위험하게 적거나 많은 용량을 처방하지 않았는지 확인하는 안전장치
  - 처방집 id로 권장 최대·최소 용량을 조회
  - 범위 밖이면 '너무 많음/적음' 오류 메시지, 범위 안이면 'Confirm' 버튼 활성화

필기 ✍️

💡 이해 스토리 1장 ⟶ 태스크 여러 장으로 쪼갠 예

Agile development techniques : XP programming

XP programming

Refactoring (1)

- Conventional wisdom in software engineering is to design for change.
- It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes **constant code improvement (refactoring)** to make changes easier when they have to be implemented.

Chapter 2 Agile Software Development 31

리팩토링 ①

- SE의 통념: **변경을 대비해 설계**하라 — 변경을 미리 예상하면 나중 비용이 준다
- XP는 반대: 변경은 **믿을 만하게 예측할 수 없으니** 그건 가치가 없다
- 대신 **끊임없는 코드 개선(리팩토링)**으로 변경이 필요할 때 쉽게 하자

필기 ✍️

★ 시험 전통 SE: design for change(예측) vs XP: refactoring(언제든 고치기 쉽게)

Agile development techniques : XP programming

XP programming

Refactoring (2)

- Programming team look for **possible software improvements** and make these improvements even where there is **no immediate need for them**.
- This **improves the understandability** of the software and so **reduces** the need for **documentation**.
- **Changes** are easier to make because the code is **well-structured and clear.**
- However, **some changes** requires **architecture refactoring** and this is much more **expensive**.

Chapter 2 Agile Software Development 32

리팩토링 ②

- 당장 필요 없어도 개선점을 찾아 개선
- SW 이해도가 높아져 **문서 필요가 준다**
- 코드가 잘 구조화돼 변경이 쉬워진다
- 하지만 어떤 변경은 **아키텍처 리팩토링**이 필요하고, 이는 **훨씬 비싸다**

필기 ✍️

⚠ 주의 리팩토링의 한계: 아키텍처 수준 변경은 비쌈

Agile development techniques : XP programming

XP programming

Examples of refactoring

- Re-organization of a class hierarchy to remove duplicate code.
- Tidying up and renaming attributes and methods to make them easier to understand.
- The replacement of inline code with calls to methods that have been included in a program library.

Chapter 2 Agile Software Development 33

리팩토링의 예

- **클래스 계층 재구성**으로 중복 코드 제거
- 속성·메서드 정리와 **이름 바꾸기**로 이해하기 쉽게
- 인라인 코드를 **라이브러리 메서드 호출로 교체**

필기 ✍️

✏ 예.풀이 예 3개 그대로 외우기

Agile development techniques : XP programming

Test development

Test-first development

- Testing is central to XP and XP has developed an approach where the program is **tested after every change** has been made.

- **XP testing features:**
  - Test-first development.
  - Incremental test development from scenarios.
  - User involvement in test development and validation.
  - Automated test harnesses are used to run all component tests each time that a new release is built.

Chapter 2 Agile Software Development 34

테스트 우선 개발 (Test-first development)

- XP의 중심은 테스트 — **변경할 때마다** 프로그램을 테스트
- XP 테스트의 특징
  - **테스트 우선** 개발
  - 시나리오에서 **점진적으로** 테스트 개발
  - 테스트 개발·검증에 **사용자 참여**
  - 새 릴리스를 빌드할 때마다 **자동 테스트 하네스**로 모든 컴포넌트 테스트 실행

필기 ✍️

★ 시험 XP 테스트 특징 4개

Agile development techniques : XP programming

Test development

Test-driven development

- Writing **tests before code** clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically.
- The test includes a check that it has executed correctly.
  - Usually relies on a testing framework such as Junit (https://junit.org/junit5/).
- All previous and **new tests** are **run automatically** when new functionality is added, thus checking that the new functionality has not introduced errors.

Chapter 2 Agile Software Development 35

테스트 주도 개발 (Test-driven development)

- 코드보다 테스트를 먼저 쓰면 **구현할 요구가 명확해진다**
- 테스트를 데이터가 아닌 **프로그램**으로 작성 → 자동 실행
- 테스트에 **올바르게 실행됐는지 검사**가 들어 있다
  - 보통 **JUnit** 같은 테스트 프레임워크 사용
- 새 기능을 넣으면 **이전 + 새 테스트 모두** 자동 실행 → 새 기능이 오류를 만들지 않았는지 확인

필기 ✍️

💡 이해 이전 테스트도 다시 돌린다 = 회귀 테스트 효과

Agile development techniques : XP programming

Test development

Customer involvement

- The **role of the customer** in the testing process is to help develop **acceptance tests** for the stories that are to be implemented in the next release of the system.
- The **customer** who is part of the team writes tests as development proceeds.
  - All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the **customer** role have **limited time** available and so cannot work full-time with the development team.
  - They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

Chapter 2 Agile Software Development 36

테스트에서의 고객 참여

- 고객의 역할: 다음 릴리스에 구현할 스토리의 **인수 테스트(acceptance test)** 개발을 돕는 것
- 팀의 일원인 고객이 개발과 함께 테스트를 작성 → 모든 새 코드가 고객이 필요한 것인지 검증
- **문제**: 고객 역할을 맡은 사람은 시간이 부족해 풀타임 참여가 어렵다
  - 요구를 준 것으로 충분하다고 여겨 테스트 참여를 꺼릴 수 있다

필기 ✍️

★ 시험 고객 ⟶ 인수 테스트 / 한계: 시간 부족·참여 꺼림

Agile development techniques : XP programming

Test development

[Test case description for dose checking]

Test 4: Dose checking
Input: 1. A number in mg representing a single dose of the drug. 2. A number representing the number of single doses per day. **Tests:** 1. Test for inputs where the single dose is correct but the frequency is too high. 2. Test for inputs where the single dose is too high and too low. 3. Test for inputs where the single dose * frequency is too high and too low. 4. Test for inputs where single dose * frequency is in the permitted range. **Output:** OK or error message indicating that the dose is outside the safe range.

Chapter 2 Agile Software Development 37

예: 용량 검사 테스트 케이스

- Test 4: 용량 검사
- **입력**: 1회 용량(mg), 하루 복용 횟수
- **테스트**
  - 1) 1회 용량은 맞지만 횟수가 너무 많을 때
  - 2) 1회 용량이 너무 많을 때와 너무 적을 때
  - 3) 1회 용량 × 횟수가 너무 많을 때와 너무 적을 때
  - 4) 1회 용량 × 횟수가 허용 범위 안일 때
- **출력**: OK 또는 '안전 범위 밖' 오류 메시지

필기 ✍️

💡 이해 p.30 Task 3을 테스트로 바꾼 것 — 경계·정상 케이스를 다 덮음

Agile development techniques : XP programming

Test development

Test automation

- **Test automation** means that tests are **written as executable components** before the task is implemented
  - These testing components should be stand-alone, should simulate the submission of input to be tested and should check that the result meets the output specification.
  - An automated test framework (e.g. JUnit) is a system that makes it easy to write executable tests and submit a set of tests for execution.

- As testing is automated, there is **always a set of tests** that can be **quickly** and **easily** executed
  - Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

Chapter 2 Agile Software Development 38

테스트 자동화

- 태스크 구현 **전에** 실행 가능한 컴포넌트로 테스트 작성
  - 독립 실행형, 입력 제출을 흉내 내고 결과가 출력 명세에 맞는지 검사
- 자동 테스트 프레임워크(예: **JUnit**) = 실행 가능한 테스트를 쉽게 쓰고 묶어서 실행하게 해 주는 시스템
- 자동화돼 있으니 언제나 **빠르고 쉽게** 돌릴 수 있는 테스트 세트가 있다
  - 기능을 추가할 때마다 돌려 새 코드가 만든 문제를 **즉시** 잡음

필기 ✍️

— 넘어가도 되는 쪽

Agile development techniques : XP programming

Test development

Problems with test-first development

- Programmers prefer programming to testing and sometimes they **take short cuts** when writing tests.
  - For example, they may write incomplete tests that do not check for all possible exceptions that may occur.
- Some tests can be **very difficult** to write incrementally.
  - For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the 'display logic' and workflow between screens.
- It **difficult** to judge the **completeness** of a set of tests.
  - Although you may have a lot of system tests, your test set may not provide complete coverage.

Chapter 2 Agile Software Development 39

테스트 우선 개발의 문제

- 프로그래머는 테스트보다 프로그래밍을 좋아해 테스트를 **대충** 쓰기도 한다
  - 예: 가능한 예외를 다 검사하지 않는 불완전한 테스트
- 어떤 테스트는 **점진적으로 쓰기 매우 어렵다**
  - 예: 복잡한 UI의 '화면 표시 로직'과 화면 간 흐름
- 테스트 세트의 **완전성을 판단하기 어렵다**
  - 테스트가 많아도 **커버리지**가 완전하지 않을 수 있다

필기 ✍️

★ 시험 문제 3개: 대충 씀 · UI 등 점진 작성 어려움 · 완전성 판단 어려움

Agile development techniques : XP programming

Pair programming (1)

- Pair programming involves **programmers** working in **pairs**, developing code together.
- This helps develop **common ownership of code** and **spreads knowledge** across the team.
- It serves as an informal **review process** as each line of code is looked at by more than 1 person.
- It **encourages refactoring** as the whole team can benefit from improving the system code.

Chapter 2 Agile Software Development 40

짝 프로그래밍 ①

- 프로그래머가 **둘씩** 함께 코드를 개발
- 코드 **공동 소유**를 키우고 팀 전체에 **지식을 퍼뜨림**
- 모든 코드 줄을 2명 이상이 보니 **비공식 검토(informal review)** 역할
- 팀 전체가 코드 개선의 이득을 보니 **리팩토링을 장려**

필기 ✍️

★ 시험 장점: 공동 소유 · 지식 공유 · 비공식 리뷰 · 리팩토링 장려

Agile development techniques : XP programming

Pair programming (2)

- In pair programming, programmers sit together at the **same computer to develop** the software.
- Pairs are created dynamically so **that all team members work with each other** during the development process.
- The **sharing of knowledge** that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.
- But, sometimes, it is inefficient if high-level programmers work together

Chapter 2 Agile Software Development 41

짝 프로그래밍 ②

- **같은 컴퓨터** 앞에 함께 앉아 개발
- 짝은 **동적으로** 바뀜 — 모든 팀원이 서로 한 번씩 함께 일함
- 지식 공유가 중요한 이유: 팀원이 **떠날 때 프로젝트 위험을 줄인다**
- 짝 프로그래밍이 꼭 비효율적이진 않다 — 짝이 따로 일하는 2명보다 효율적이라는 근거도 있음
- 하지만 **고급 개발자끼리** 짝이면 비효율적일 때도

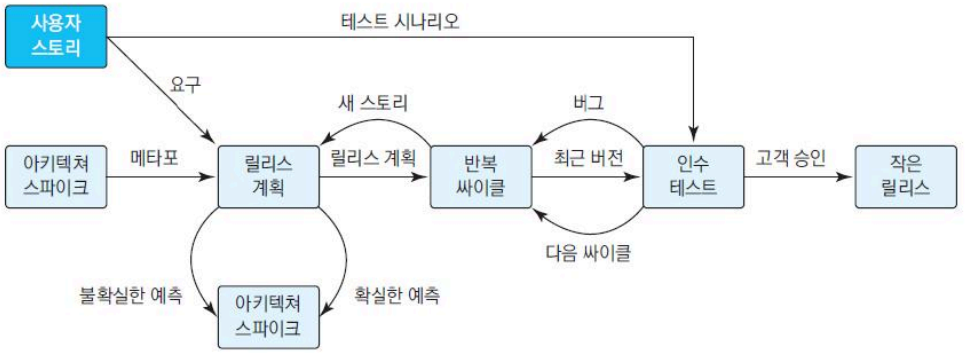
필기 ✍️

⚠ 주의 '짝 프로그래밍은 항상 2배 비효율' → X

Agile development techniques : XP programming

Extreme programming flow

[Extreme programming flow (as Korean)]



Source: <All about software engineering>. 생능출판사. 최은만

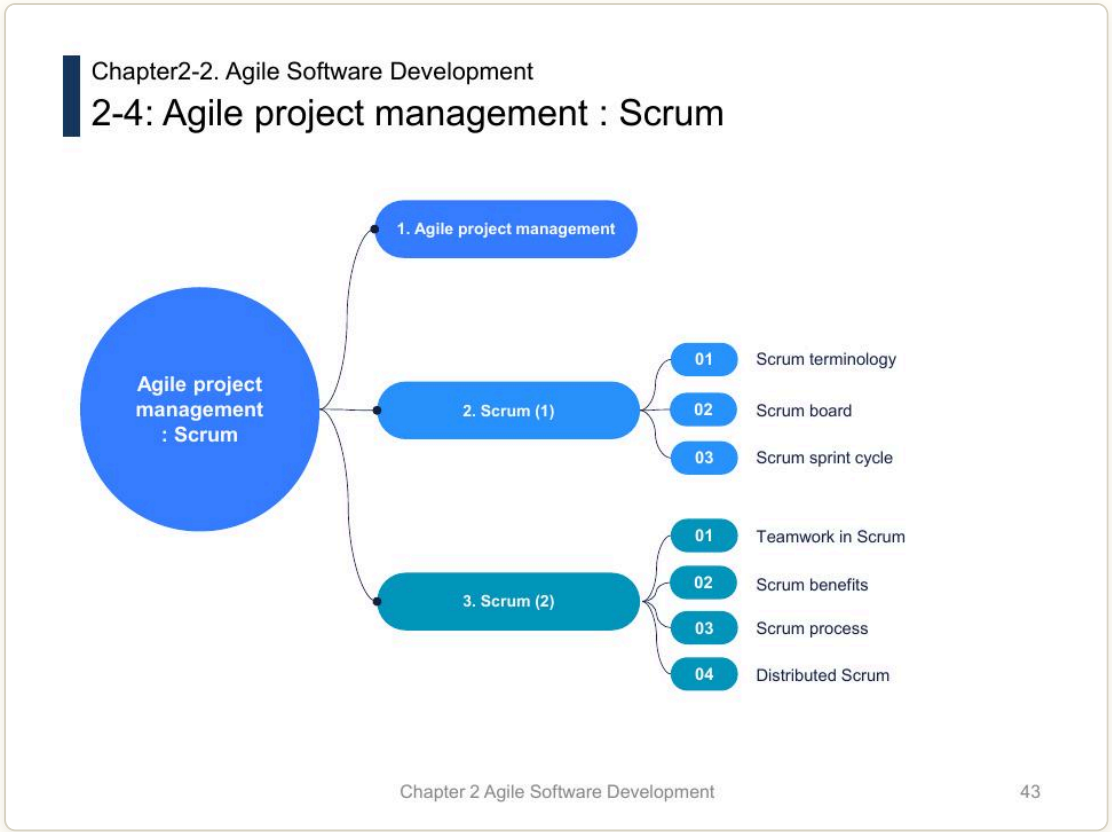
Chapter 2 Agile Software Development 42

XP 흐름 (한국어 그림)

- 사용자 스토리 → (요구) → **릴리스 계획** ← (메타포) 아키텍처 스파이크
  - 불확실한 예측이면 아키텍처 스파이크로 확인 → 확실한 예측
- 릴리스 계획 → **반복 싸이클** (새 스토리·다음 싸이클로 반복) → 최근 버전 → **인수 테스트** (버그면 되돌아감)
- 사용자 스토리 → (테스트 시나리오) → 인수 테스트 → 고객 승인 → **작은 릴리스**

필기 ✍️

💡 이해 스파이크 = 불확실한 걸 확인하려고 짧게 만들어 보는 실험



2-4: 애자일 프로젝트 관리 — 스크럼

- 1. 애자일 프로젝트 관리
- 2. 스크럼 ① 용어, 스크럼 보드, 스프린트 주기
- 3. 스크럼 ② 팀워크, 장점, 프로세스, 분산 스크럼

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Agile project management : Scrum

Agile project management

- The principal responsibility of software project managers is to manage the project so that the software is **delivered on time** and within the **planned budget** for the project.
- The standard approach to project management is plan-driven.
- Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- **Agile project management** requires a different approach, which is adapted to **incremental development** and the practices **used in agile methods.**

Chapter 2 Agile Software Development 44

애자일 프로젝트 관리

- SW 프로젝트 관리자의 주된 책임: SW를 **제때, 예산 안에서** 전달하도록 관리
- 표준 방식은 **계획 주도** — 무엇을, 언제, 누가 만들지 계획을 세움
- 애자일 프로젝트 관리는 **다른 접근**이 필요 — 점증적 개발과 애자일 실천에 맞춘 방식

필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (1)

Scrum

Scrum is an **agile method** that focuses on managing **iterative development** rather than specific agile practices.

There are three phases in Scrum.

- **The initial phase** is an **outline planning** phase where you establish the general **objectives** for the project and **design** the software architecture.
- This is followed by a series of **sprint cycles**, where each cycle develops an **increment** of the system.
- The **project closure phase** wraps up the project, **completes** required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Chapter 2 Agile Software Development 45

스크럼 (Scrum)

- 특정 애자일 실천보다 **반복 개발의 관리**에 초점을 둔 애자일 방법
- **3단계**
  - **초기 단계** — 개략 계획: 프로젝트 전체 목표 설정, SW 아키텍처 설계
  - **스프린트 주기**의 연속 — 주기마다 시스템 증분 하나 개발
  - **프로젝트 종료 단계** — 마무리, 도움말·사용자 매뉴얼 같은 필요 문서 완성, 교훈 평가

필기 ✍️

★ 시험 스크럼 3단계: 개략 계획 ⟶ 스프린트 주기들 ⟶ 종료

Agile project management : Scrum

Scrum (1)

Scrum terminology

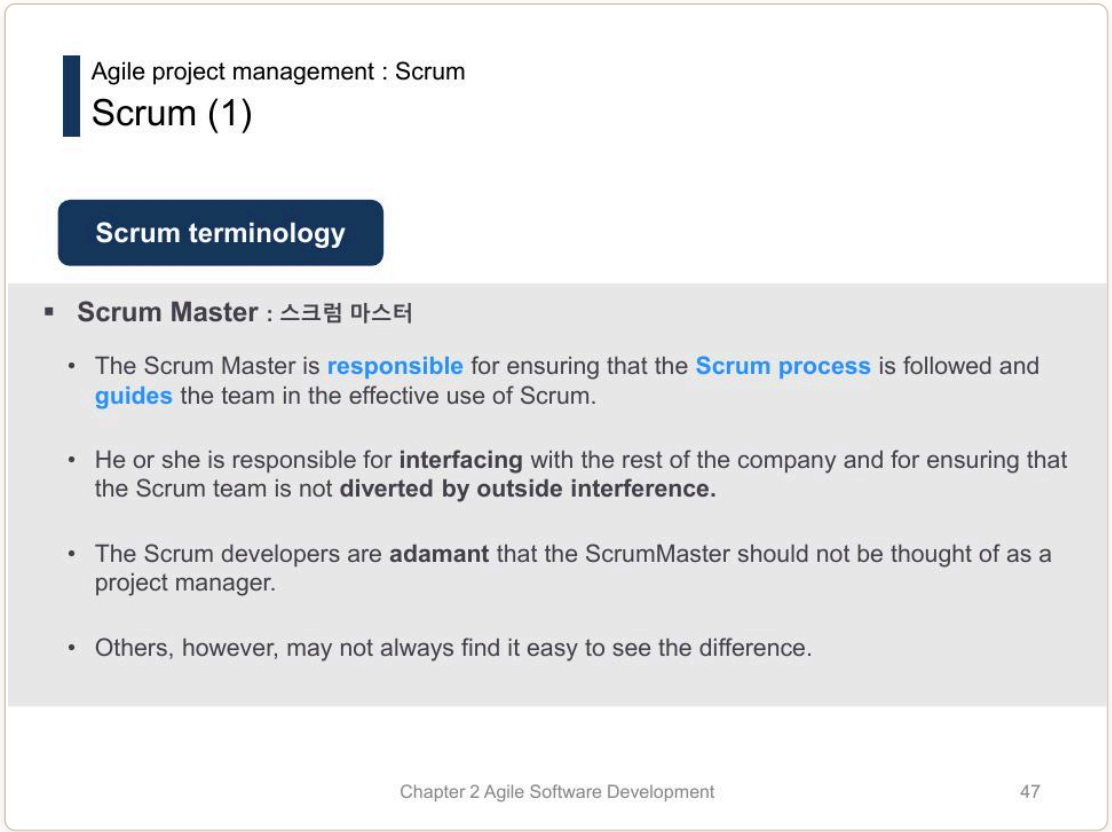
- **Development team** : 개발팀
  - A **self-organizing group** of software developers, which should be no more than 7 people.
  - They are responsible for **developing the software** and other **essential project documents**.
- **Sprint** : 스프린트
  - A development iteration.
  - Sprints are usually 2-4 weeks long.
- **Scrum** : 스크럼
  - A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day.
  - Ideally, this should be a short face-to-face meeting that includes the whole team.

스크럼 용어 ①

- **Development team (개발팀)** — **자기 조직화** 개발자 그룹, **7명 이하**
  - SW와 필수 프로젝트 문서 개발 책임
- **Sprint (스프린트)** — 개발 반복 한 번, 보통 **2~4주**
- **Scrum (스크럼)** — 진행을 검토하고 그날 할 일의 우선순위를 정하는 **매일 회의**
  - 이상적으론 팀 전체가 참여하는 **짧은 대면 회의**

필기 ✍️

★ 시험 숫자: 팀 ≤ 7명 · 스프린트 2~4주 · 스크럼 = 매일 회의



스크럼 용어 ② 스크럼 마스터

- **Scrum Master** — 스크럼 프로세스를 지키게 하고, 팀이 스크럼을 효과적으로 쓰도록 안내
  - 회사의 나머지와의 **창구**, 팀이 **외부 방해로 흔들리지 않게** 함
  - 스크럼 개발자들은 스크럼 마스터를 **프로젝트 관리자로 보면 안 된다**고 강하게 주장
  - 다른 사람들은 그 차이를 알기 어렵기도

필기 ✍️

⚠ 주의 스크럼 마스터 ≠ 프로젝트 관리자 (단골 OX)

Agile project management : Scrum

Scrum (1)

Scrum terminology

- **Potentially shippable product increment** : 잠재적 전달 가능한 제품 증가분
  - The **software increment** that is delivered from a sprint.
  - The idea is that this should be 'potentially shippable' which means that it is in a **finished state and no further work**, such as testing, is needed to incorporate it into the final product.
  - In practice, this is not always achievable.
- **Product backlog** : 제품 백로그
  - This is a list of **'to do' items** which the Scrum team must tackle.
  - They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.

Chapter 2 Agile Software Development 48

스크럼 용어 ③ 증가분 · 백로그

- **Potentially shippable product increment (잠재적 전달 가능한 제품 증가분)**
  - 스프린트에서 전달되는 SW 증분
  - '잠재적 전달 가능' = **완성 상태**라 테스트 같은 추가 작업 없이 최종 제품에 넣을 수 있다
  - 실제로는 늘 가능하진 않다
- **Product backlog (제품 백로그)** — 스크럼 팀이 처리할 **'할 일' 목록**
  - 기능 정의, SW 요구, 사용자 스토리, 아키텍처 정의·사용자 문서 같은 보조 작업

필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (1)

Scrum terminology

- **Product owner** : 제품 소유권자
  - An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously **review the product backlog** to ensure that the project continues to meet critical business needs.
  - The Product Owner can be a **customer** but might also be a **product manager** in a software company or **other stakeholder** representative.

- **Velocity** : 속도
  - An estimate of how much product backlog effort that a team can cover in a single sprint.
  - Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Chapter 2 Agile Software Development 49

스크럼 용어 ④ 제품 소유자 · 속도

- **Product owner (제품 소유권자)** — 제품 기능·요구를 찾고, 개발 **우선순위**를 정하고, 백로그를 계속 검토해 핵심 비즈니스 요구를 맞추게 하는 사람(또는 소그룹)
  - 고객일 수도, SW 회사의 제품 관리자나 다른 이해관계자 대표일 수도
- **Velocity (속도)** — 팀이 **한 스프린트에 처리할 수 있는 백로그 노력량** 추정치
  - 스프린트에 무엇을 넣을지 추정하고 성과 향상을 재는 기준

필기 ✍️

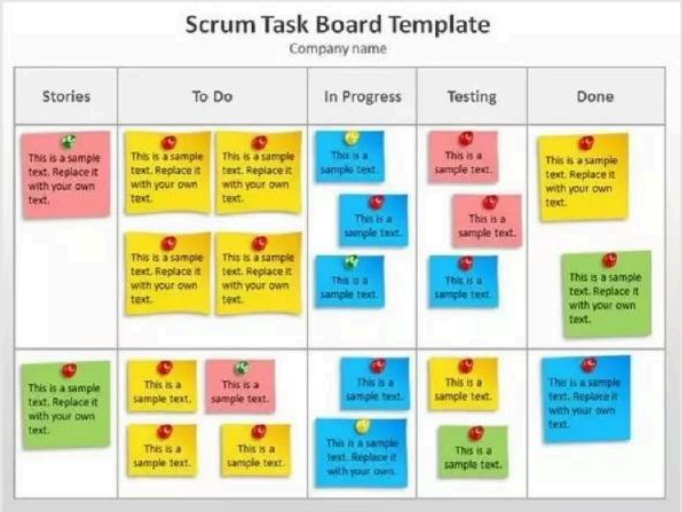
★ 시험 우선순위 정하는 사람 = Product owner (스크럼 마스터 아님!)

Agile project management : Scrum

Scrum (1)

Scrum board

- Story (backlog)
- To do
- In progress
- Testing
- Done



Source: <https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=ijoos&logNo=221463849700>

Chapter 2 Agile Software Development 50

스크럼 보드

- 열: **Story(백로그) → To do → In progress → Testing → Done**
- 카드를 왼쪽에서 오른쪽으로 옮기며 진행 상황을 보이게 함

필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (1)

[Example of scrum board]



Source: <https://scrumexplainer.com/scrum/scrum-board/>

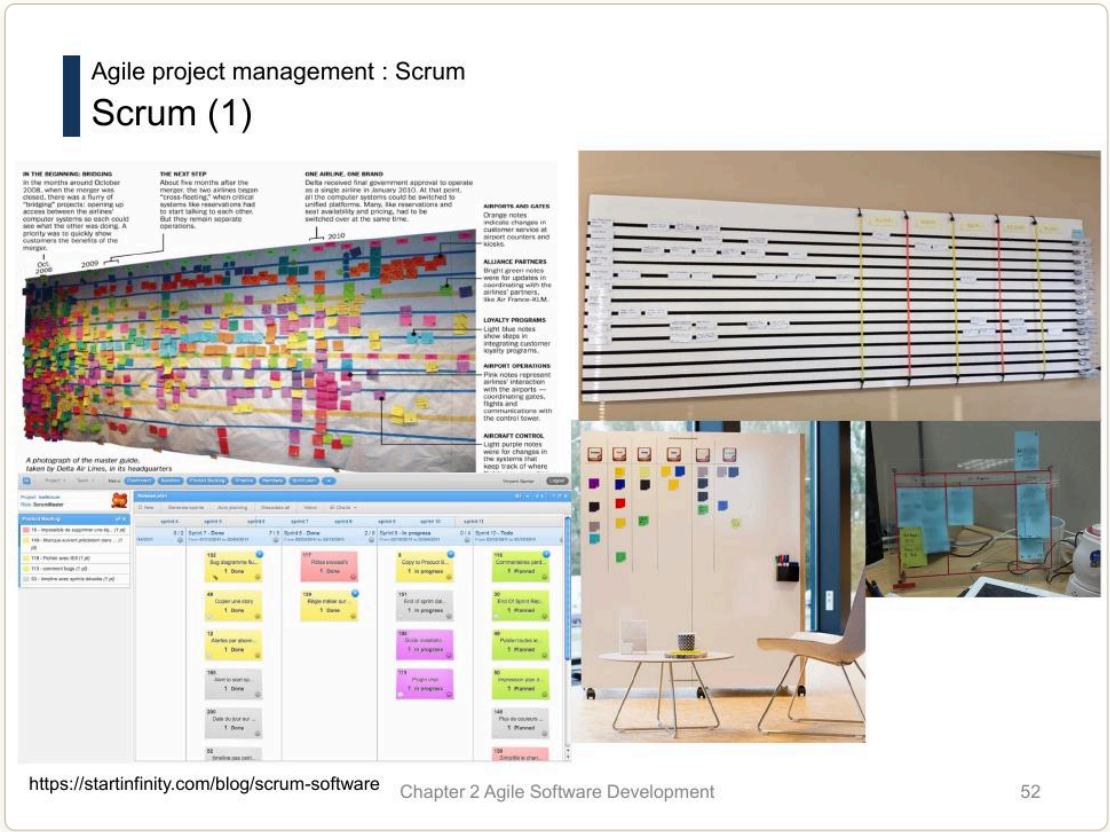
Chapter 2 Agile Software Development 51

스크럼 보드 예 (사진)

- 실제 스크럼 보드 사진 — 열마다 포스트잇 카드

필기 ✍️

— 넘어가도 되는 쪽



스크럼 소프트웨어 예 (화면)

- 스크럼 관리 도구 화면 (startinfinity.com 블로그)

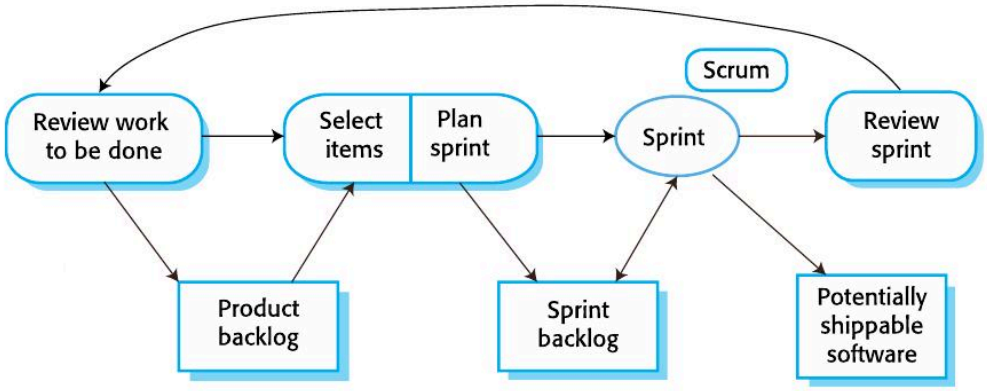
필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (1)

[Scrum sprint cycle]



Chapter 2 Agile Software Development 53

스크럼 스프린트 주기 (그림)

- 할 일 검토 → **항목 선택 + 스프린트 계획** → **스프린트**(매일 Scrum) → 스프린트 검토 → 다시 할 일 검토
  - 제품 백로그 → 항목 선택에 쓰임 / 스프린트 백로그 ↔ 스프린트 / 결과 = 잠재적 전달 가능 SW

필기 ✍️

★ 시험 제품 백로그(전체) vs 스프린트 백로그(이번 스프린트분) 구분

Agile project management : Scrum

Scrum (1)

Scrum sprint cycle (cont.)

- Sprints are fixed length, normally **2–4 weeks**.
- The **starting point** for planning is the **product backlog**, which is the list of work to be done on the project.
- The **selection phase** involves all of the project team who work **with the customer** to **select the features** and **functionality** from the product backlog to be developed during the sprint.
- Once these are agreed, the team **organize** themselves to **develop** the software.

Chapter 2 Agile Software Development 54

스프린트 주기 설명 ①

- 스프린트는 **고정 길이**, 보통 **2~4주**
- 계획의 출발점은 **제품 백로그** (프로젝트의 할 일 목록)
- **선택 단계**: 프로젝트 팀 전체가 **고객과 함께** 이번 스프린트에 개발할 기능을 백로그에서 고름
- 합의되면 팀이 **스스로 조직**해 개발

필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (1)

Scrum sprint cycle (cont.)

- During this stage the team is isolated from the customer and the organization, with all **communications** channelled through the so-called '**Scrum master**'.
- The role of the **Scrum master** is to **protect** the development team from external distractions.
- At the end of the sprint, the **work done is reviewed** and presented to stakeholders.
- The next sprint cycle then begins.

Chapter 2 Agile Software Development 55

스프린트 주기 설명 ②

- 이 단계 동안 팀은 고객·조직과 **격리**되고, 모든 소통은 **스크럼 마스터**를 거친다
- 스크럼 마스터의 역할: 개발팀을 **외부 방해에서 보호**
- 스프린트 끝에 한 일을 **검토**하고 이해관계자에게 **발표**
- 그리고 다음 스프린트 시작

필기 ✍️

★ 시험 스프린트 중 외부 소통 창구 = 스크럼 마스터

Agile project management : Scrum

Scrum (2)

Teamwork in Scrum

- **The 'Scrum master' is a facilitator**
  - arranges daily meetings
  - tracks the backlog of work to be done
  - records decisions
  - measures progress against the backlog and communicates with customers and management outside of the team.
- **The whole team attends short daily meetings (Scrums)**
  - all team members share information
  - describe their progress since the last meeting
  - problems that have arisen
  - what is planned for the following day.
  - everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Chapter 2 Agile Software Development 56

스크럼의 팀워크

- 스크럼 마스터는 **촉진자(facilitator)**
  - 매일 회의 준비, 할 일 백로그 추적, 결정 기록
  - 백로그 대비 진척 측정, 팀 밖 고객·경영진과 소통
- 팀 전체가 짧은 **매일 회의(Scrum)**에 참석
  - 지난 회의 이후 진척, 생긴 문제, 다음 날 계획을 공유
  - 모두가 상황을 알고, 문제가 생기면 단기 작업을 **다시 계획**

필기 ✍️

💡 이해 매일 회의 3질문: 한 일 · 문제 · 할 일

Agile project management : Scrum

Scrum (2)

Scrum benefits

- The product is **broken down** into a set of **manageable and understandable chunks**.
- **Unstable requirements** do **not hold** up progress.
- The whole team have **visibility of everything** and consequently team **communication is improved**.
- **Customers** see **on-time** delivery of **increments** and gain **feedback** on how the product works.
- **Trust between customers and developers** is established and a positive culture is created in which everyone expects the project to succeed.

Chapter 2 Agile Software Development 57

스크럼의 장점

- 제품을 **관리 가능하고 이해하기 쉬운 덩어리**로 나눔
- **불안정한 요구**가 진행을 막지 않음
- 팀 전체가 모든 것을 볼 수 있어 **팀 소통 향상**
- 고객이 증분의 **제때 전달**을 보고 피드백
- 고객과 개발자 사이 **신뢰** 형성, 모두 성공을 기대하는 긍정적 문화

필기 ✍️

★ 시험 장점 5개

Agile project management : Scrum

Scrum (2)

[Scrum process (as Korean)]



Source: <All about software engineering>. 생능출판사. 최은만

Chapter 2 Agile Software Development 58

스크럼 프로세스 (한국어 그림)

- **프로덕트 백로그** → **스프린트 백로그** → 스프린트 **2~4주** (매일 **24시간**마다 스크럼 미팅) → **배포 가능 프로덕트 추가분**

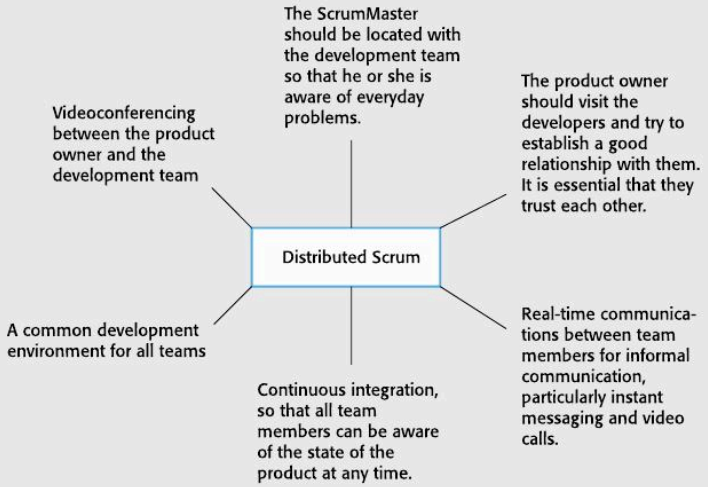
필기 ✍️

— 넘어가도 되는 쪽

Agile project management : Scrum

Scrum (2)

[Distributed Scrum]



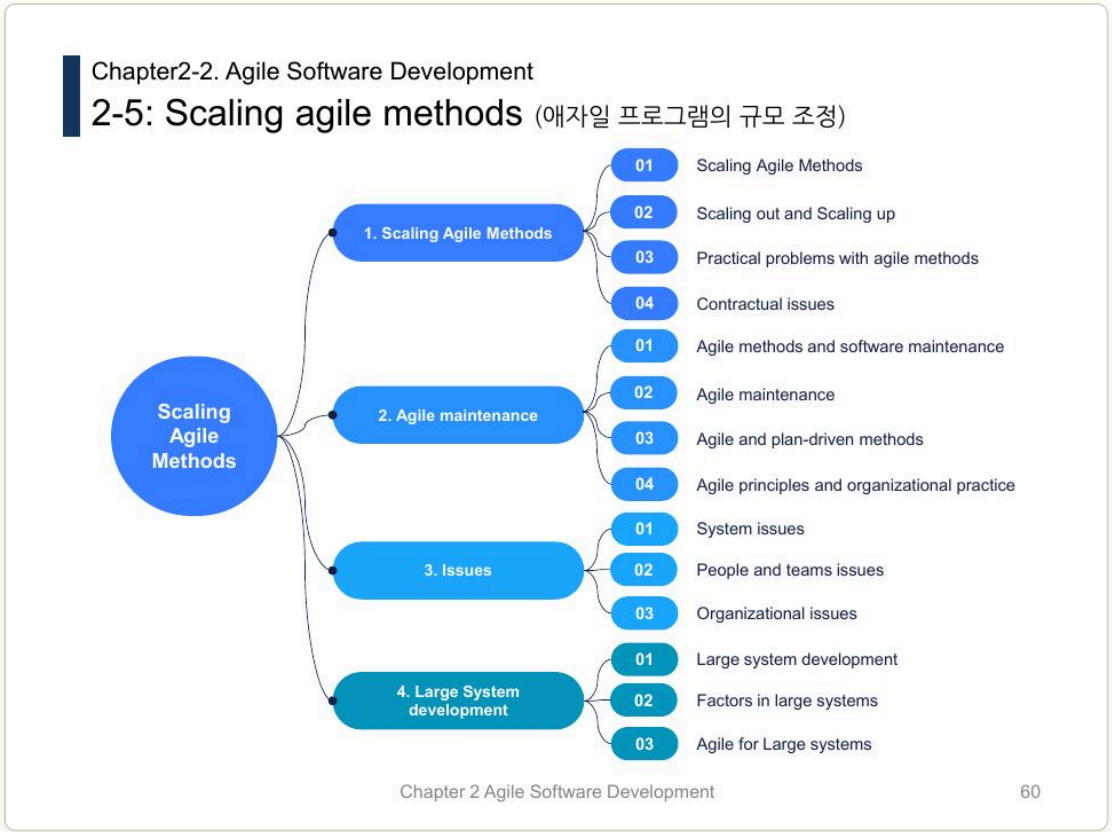
Chapter 2 Agile Software Development 59

분산 스크럼 (Distributed Scrum)

- 스크럼 마스터는 **개발팀과 같은 곳**에 있어야 — 일상 문제를 알 수 있게
- 제품 소유자는 개발자를 **방문**해 좋은 관계를 쌓아야 — 서로 신뢰가 필수
- 제품 소유자와 개발팀 사이 **화상 회의**
- 모든 팀에 **공통 개발 환경**
- **연속적 통합** — 모두가 언제든 제품 상태를 알게
- 비공식 소통을 위한 **실시간 소통**(메신저, 화상 통화)

필기 ✍️

💡 이해 흩어진 팀일수록 '소통'을 도구로 보강한다는 게 요지

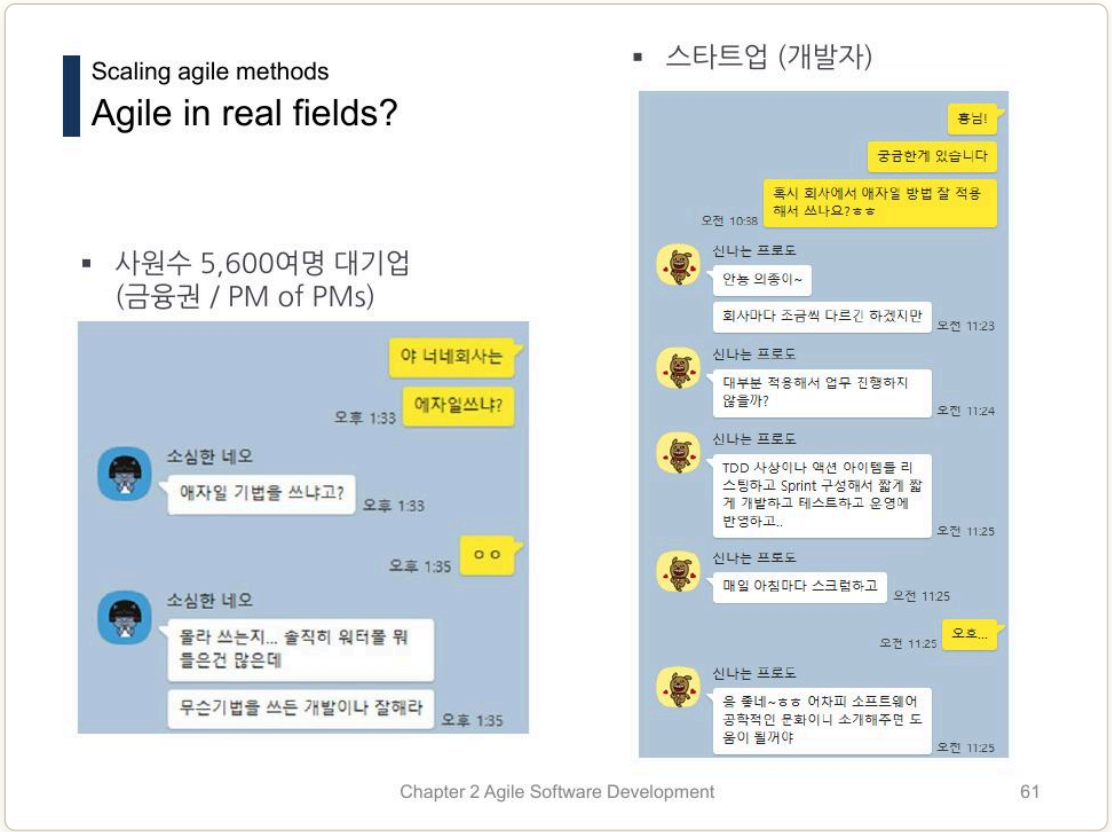


2-5: 애자일 방법의 규모 조정

- 1. 규모 조정 — scaling up / scaling out, 실제 문제, 계약 문제
- 2. 애자일 유지보수 — 애자일과 계획 주도, 원칙과 조직 관행
- 3. 이슈 — 시스템·사람과 팀·조직
- 4. 대규모 시스템 개발 — 요인, 대규모 애자일

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도



현장에서 애자일은? (메신저 대화)

- 스타트업 개발자, 금융권 대기업(5,600명) PM에게 '회사에서 애자일 쓰나?' 물어본 대화 캡처
- 요지: 회사·팀마다 다르다 — 스크럼·스프린트를 쓰는 곳도, 사실상 폭포수인 곳도, 둘을 섞은 곳도 있다

필기 ✍️

💡 이해 현실은 '계획 주도 + 애자일 혼합' (P.84 핵심 정리와 같은 말)

Scaling agile methods

Agile in real fields?

- 사원수 4,200여명 대기업 (개발자)



- 사원수 11,100여명 대기업 (연구직)



Chapter 2 Agile Software Development 62

현장에서 애자일은? (메신저 대화)

- 대기업(4,200명) 개발자, 대기업(11,100명) 연구직에게 '회사에서 애자일 쓰나?' 물어본 대화 캡처
- 요지: 회사·팀마다 다르다 — 스크럼·스프린트를 쓰는 곳도, 사실상 폭포수인 곳도, 둘을 섞은 곳도 있다

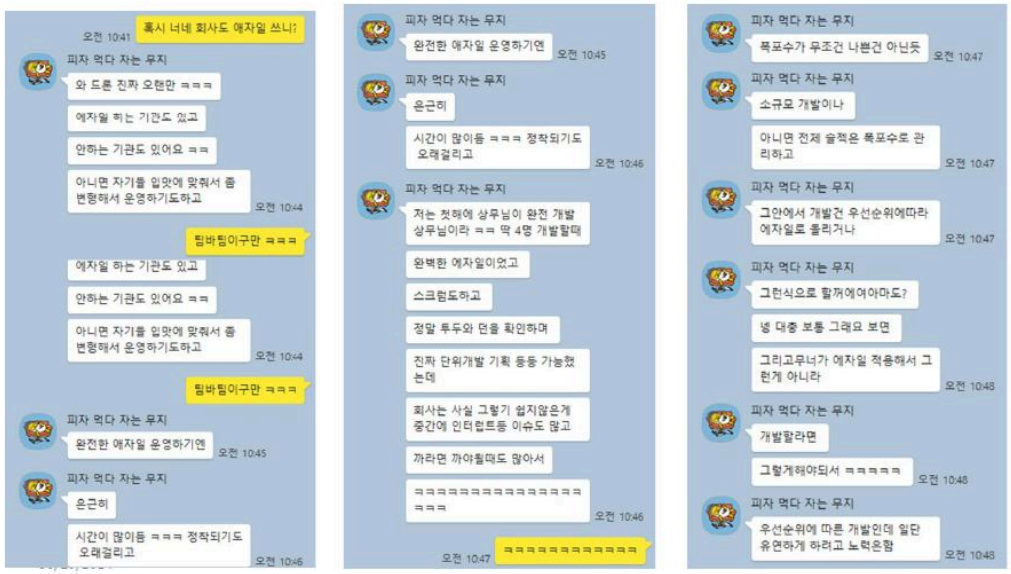
필기 ✍️

💡 이해 현실은 '계획 주도 + 애자일 혼합' (P.84 핵심 정리와 같은 말)

Scaling agile methods

Agile in real fields?

(사원수 22,300여명 대기업 / 연구직)



Chapter 2 Agile Software Development 63

현장에서 애자일은? (메신저 대화)

- 대기업(22,300명) 연구직에게 '회사에서 애자일 쓰나?' 물어본 대화 캡처
- 요지: 회사·팀마다 다르다 — 스크럼·스프린트를 쓰는 곳도, 사실상 폭포수인 곳도, 둘을 섞은 곳도 있다

필기 ✍️

💡 이해 현실은 '계획 주도 + 애자일 혼합' (P.84 핵심 정리와 같은 말)



현장에서 애자일은? (메신저 대화)

- 대기업(22,300명) 연구직 (이어서)에게 '회사에서 애자일 쓰나?' 물어본 대화 캡처
- 요지: 회사·팀마다 다르다 — 스크럼·스프린트를 쓰는 곳도, 사실상 폭포수인 곳도, 둘을 섞은 곳도 있다

필기 ✍️

💡 이해 현실은 '계획 주도 + 애자일 혼합' (P.84 핵심 정리와 같은 말)

Scaling agile methods

Agile in real fields?

(사원수 1200명 중견기업)



Chapter 2 Agile Software Development 65

현장에서 애자일은? (메신저 대화)

- 중견기업(1,200명)에게 '회사에서 애자일 쓰나?' 물어본 대화 캡처
- 요지: 회사·팀마다 다르다 — 스크럼·스프린트를 쓰는 곳도, 사실상 폭포수인 곳도, 둘을 섞은 곳도 있다

필기 ✍️

💡 이해 현실은 '계획 주도 + 애자일 혼합' (P.84 핵심 정리와 같은 말)

Scaling agile methods

Scaling agile methods

[Scaling agile methods]

- Agile methods have proved to be successful for **small and medium sized projects** that can be developed by a small co-located team.
- It is sometimes argued that the **success of these methods** comes because of **improved communications** which is possible **when everyone is working together**.
- Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

Chapter 2 Agile Software Development 66

애자일 방법의 규모 조정

- 애자일은 **한곳에 모인 작은 팀**이 하는 **중소 규모** 프로젝트에서 성공해 왔다
- 성공 이유는 모두 함께 일해서 가능한 **소통 향상** 덕분이라는 주장도 있다
- **Scaling up** = 여러 팀이, 아마 여러 장소에서 일하는 **더 크고 긴** 프로젝트에 맞게 애자일을 바꾸는 것

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Scaling agile methods

[Scaling out and scaling up]

- **'Scaling up'**
  - Concerned with using agile methods for developing large software systems that cannot be developed by a small team.
- **'Scaling out'**
  - concerned with how agile methods can be introduced across a large organization with many years of software development experience.
- **When scaling agile methods it is important to maintain agile fundamentals:**
  - Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

Chapter 2 Agile Software Development 67

Scaling up vs Scaling out

- **Scaling up** — 작은 팀으로는 못 만드는 **대형 SW 시스템** 개발에 애자일 쓰기
- **Scaling out** — 오랜 SW 개발 경험을 가진 **대형 조직 전체**에 애자일 도입하기
- 규모를 키워도 애자일 기본은 유지해야
  - 유연한 계획, 잦은 릴리스, 연속 통합, 테스트 주도 개발, 좋은 팀 소통

필기 ✍️

★ 시험 up = 큰 시스템 / out = 큰 조직 — 헷갈리기 쉬움!

Scaling agile methods

Scaling agile methods

[Practical problems with agile methods]

- The informality of agile development is incompatible with the **legal approach** to contract definition that is commonly used in large companies.
- Agile methods are most **appropriate** for new software **development** rather than **software maintenance**.
  - Yet the majority of software costs in large companies come from maintaining their existing software systems.
- **Agile methods** are designed for small **co-located teams** yet much software development now involves worldwide distributed teams.

Chapter 2 Agile Software Development 68

애자일의 실제 문제

- 애자일의 **비공식성**은 대기업이 흔히 쓰는 **법적 계약 정의 방식**과 맞지 않는다
- 애자일은 **새 SW 개발**에 가장 알맞고 **유지보수**엔 덜 맞다
  - 그런데 대기업 SW 비용 대부분은 기존 시스템 **유지보수**에서 나온다
- 애자일은 **한곳에 모인 작은 팀**용인데, 요즘 개발은 **전 세계 분산 팀**이 많다

필기 ✍️

★ 시험 실제 문제 3개: 계약 · 유지보수 · 분산 팀

Scaling agile methods

Scaling agile methods

[Contractual issues]

- **Most software contracts** for custom systems are **based around a specification**, which sets out what has to be implemented by the system developer for the system customer.
- However, this precludes **interleaving specification and development** as is the norm in agile development.
- A contract that **pays** for developer **time** rather than functionality is required.
  - However, this is seen as a high risk my many legal departments because what has to be delivered cannot be guaranteed.

Chapter 2 Agile Software Development 69

계약 문제

- 맞춤 시스템 계약 대부분은 **명세** 기반 — 개발자가 고객을 위해 무엇을 구현할지 정함
- 이는 애자일의 기본인 **명세와 개발의 번갈아 진행**을 막는다
- **기능이 아니라 개발자 시간에 돈을 내는** 계약이 필요
  - 하지만 무엇이 전달될지 보장할 수 없어 많은 법무팀이 **고위험**으로 본다

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Agile maintenance

[Agile methods and software maintenance]

- Most organizations **spend more on maintaining** existing software than they do on new software development.
  - So, if agile methods are to be successful, they have to support maintenance as well as original development.
- Two key issues:
  - Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of **minimizing formal documentation**?
  - Can agile methods be used **effectively for evolving a system** in response to customer change requests?
- Problems may arise if original development team cannot be maintained.

Chapter 2 Agile Software Development 70

애자일 방법과 SW 유지보수

- 대부분 조직은 새 개발보다 **기존 SW 유지보수에 더 많이** 쓴다 → 애자일이 성공하려면 유지보수도 지원해야
- 핵심 쟁점 2개
  - 공식 문서를 최소화하는 애자일로 만든 시스템이 **유지보수 가능한가?**
  - 고객 변경 요청에 맞춰 시스템을 진화시키는 데 애자일을 **효과적으로 쓸 수 있나?**
- 원래 개발팀을 유지하지 못하면 문제가 생길 수 있다

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Agile maintenance

Agile maintenance

- Key problems are:
  - **Lack** of product **documentation**
  - **Keeping customers involved** in the development process
  - **Maintaining** the continuity of the **development team**
- Agile development relies on the development team knowing and understanding what has to be done.
- For long-lifetime systems, this is a real problem as the original developers will not always work on the system.

애자일 유지보수의 핵심 문제

- **제품 문서 부족**
- 고객을 개발에 **계속 참여**시키기
- 개발팀의 **연속성** 유지
- 애자일은 팀이 할 일을 **알고 이해**하는 데 의존 → 수명이 긴 시스템에선 원래 개발자가 늘 남아 있지 않아 실제 문제

필기 ✍️

★ 시험 유지보수 문제 3개: 문서 부족 · 고객 참여 유지 · 팀 연속성

Scaling agile methods

Agile maintenance

Agile and plan-driven methods

- Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:
  - Is it important to have a very **detailed specification and design** before moving to implementation? If so, you probably need to use **a plan-driven approach**.
  - Is an **incremental delivery strategy**, where you deliver the software to customers and get **rapid feedback** from them, realistic? If so, consider **using agile methods**.

Chapter 2 Agile Software Development 72

애자일 vs 계획 주도 — 균형 정하기 ①

- 대부분 프로젝트는 두 방식의 요소를 **섞는다**. 균형은 다음에 달려 있다
  - 구현 전에 **아주 자세한 명세·설계**가 중요한가? → 그렇다면 **계획 주도**
  - SW를 고객에게 전달하고 빠른 피드백을 받는 **점증적 인도가 현실적**인가? → 그렇다면 **애자일** 고려

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Agile maintenance

Agile and plan-driven methods

- Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:
  - **How large is the system** that is being developed?
    - **Agile** methods are most effective when the system can be developed with a **small co-located team** who can communicate informally.
    - Agile may not be possible for **large systems** that require larger development teams so a **plan-driven** approach may have to be used.

Chapter 2 Agile Software Development 73

애자일 vs 계획 주도 — 균형 정하기 ②

- 개발하는 시스템이 **얼마나 큰가?**
  - 애자일은 비공식 소통이 되는 **한곳의 작은 팀**일 때 가장 효과적
  - 큰 팀이 필요한 대형 시스템엔 애자일이 안 될 수 있어 **계획 주도**를 써야 할 수도

필기 ✍️

★ 시험 자세한 명세 필요 ⟶ 계획 주도 / 점증 인도 가능 ⟶ 애자일 / 대형 ⟶ 계획 주도

Scaling agile methods

Agile maintenance

Agile principles and organizational practice

[Customer involvement : 고객 참여]

- This depends on having a customer who is willing and able to spend time with the development team and who can represent all system stakeholders.
  - Often, customer representatives have other demands on their time and cannot play a full part in the software development.
- Where there are external stakeholders, such as regulators, it is difficult to represent their views to the agile team.

[Embrace change : 변화 수용]

- Prioritizing changes can be extremely difficult, especially in systems for which there are many stakeholders.
- Typically, each stakeholder gives different priorities to different changes.

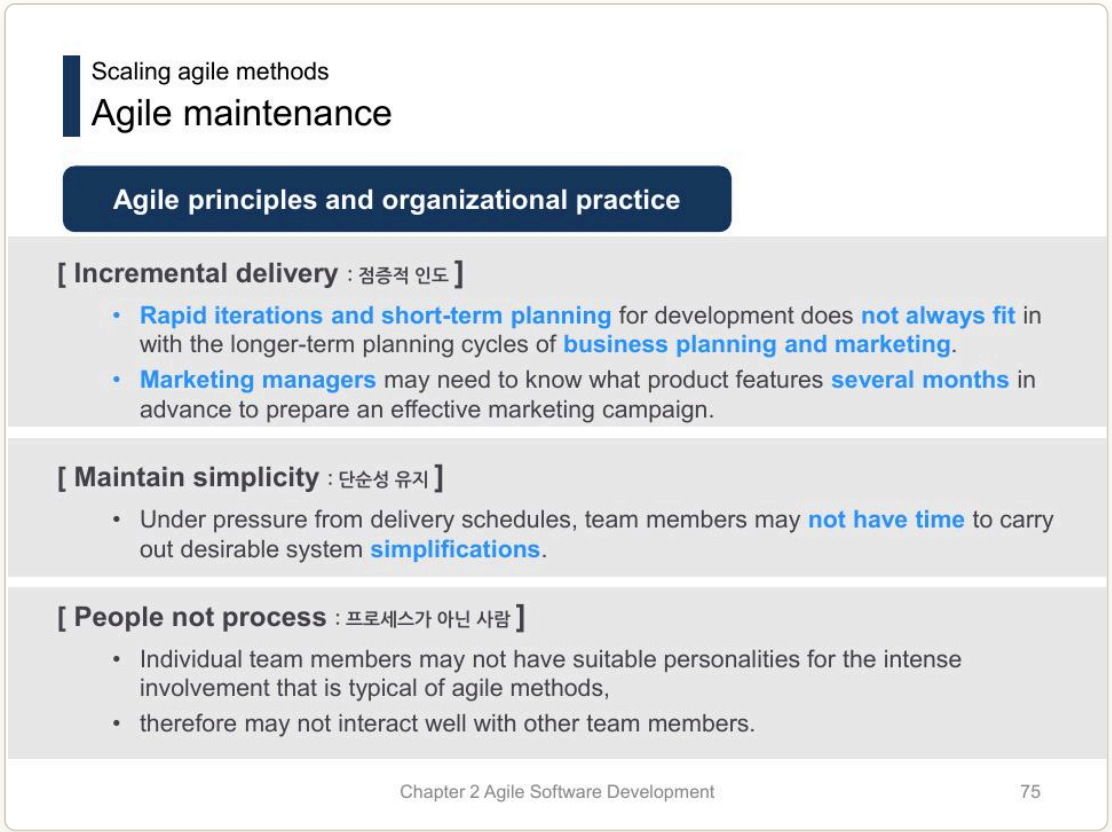
Chapter 2 Agile Software Development 74

애자일 원칙 vs 조직 현실 ①

- **고객 참여**
  - 팀과 시간을 보낼 의지·능력이 있고 **모든 이해관계자를 대표**할 고객이 있어야
  - 고객 대표는 다른 일이 많아 개발에 온전히 참여 못 하기도
  - **규제 기관** 같은 외부 이해관계자의 관점을 애자일 팀에 전달하기 어렵다
- **변화 수용**
  - 이해관계자가 많으면 변경 **우선순위 정하기**가 극히 어렵다 — 저마다 다른 우선순위

필기 ✍️

— 넘어가도 되는 쪽


Scaling agile methods

Agile maintenance

Agile principles and organizational practice

[Incremental delivery : 점증적 인도 **]**

- **Rapid iterations and short-term planning** for development does **not always fit** in with the longer-term planning cycles of **business planning and marketing**.
- **Marketing managers** may need to know what product features **several months** in advance to prepare an effective marketing campaign.

[Maintain simplicity : 단순성 유지 **]**

- Under pressure from delivery schedules, team members may **not have time** to carry out desirable system **simplifications**.

[People not process : 프로세스가 아닌 사람 **]**

- Individual team members may not have suitable personalities for the intense involvement that is typical of agile methods,
- therefore may not interact well with other team members.

Chapter 2 Agile Software Development 75

애자일 원칙 vs 조직 현실 ②

- **점증적 인도** — 빠른 반복과 단기 계획이 사업 계획·마케팅의 **장기 계획 주기**와 안 맞음
  - 마케팅 관리자는 효과적인 캠페인을 위해 몇 달 전에 기능을 알아야
- **단순성 유지** — 일정 압박으로 단순화할 **시간이 없을** 수 있다
- **프로세스가 아닌 사람** — 애자일 특유의 강한 참여에 **성격이 안 맞는** 팀원도 있어 잘 어울리지 못할 수 있다

필기 ✍️

💡 이해 P.15~17의 원칙 5개가 '현실에선 왜 어려운가'를 하나씩 짚지은 쪽

Scaling agile methods

Issues

System issues

- **How large** is the system being developed?
  - Agile methods are most effective a relatively small co-located team who can communicate informally.
- **What type of system** is being developed?
  - Systems that require a lot of analysis before implementation need a fairly detailed design to carry out this analysis.
- **What is the expected** system lifetime?
  - Long-lifetime systems require documentation to communicate the intentions of the system developers to the support team.
- Is the system subject to **external regulation**?
  - If a system is regulated you will probably be required to produce detailed documentation as part of the system safety case.

Chapter 2 Agile Software Development 76

이슈 ① 시스템

- **시스템 크기** — 애자일은 비공식 소통이 되는 작은 팀에서 가장 효과적
- **시스템 종류** — 구현 전에 분석이 많이 필요하면 꽤 자세한 설계가 필요
- **예상 수명** — 수명이 길면 개발 의도를 지원팀에 전할 **문서**가 필요
- **외부 규제** — 규제 대상이면 안전 사례(safety case)용 **상세 문서**를 내야 할 것

필기 ✍️

★ 시험 시스템 이슈 4개: 크기 · 종류 · 수명 · 규제

Scaling agile methods

Issues

People and teams issues

- How **good** are the **designers** and **programmers** in the development team?
  - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code.
- How is the **development team organized**?
  - Design documents may be required if the team is dsitributed.
- What **support technologies** are available?
  - IDE support for visualisation and program analysis is essential if design documentation is not available.

Chapter 2 Agile Software Development 77

이슈 ② 사람과 팀

- 설계자·프로그래머가 **얼마나 뛰어난가?**
  - 애자일은 상세 설계를 코드로 옮기기만 하는 계획 기반보다 **더 높은 기술 수준**이 필요하다는 주장
- 팀이 **어떻게 조직**되어 있나? — 분산돼 있으면 설계 문서가 필요할 수 있다
- 어떤 **지원 기술**이 있나? — 설계 문서가 없으면 시각화·프로그램 분석용 **IDE 지원**이 필수

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Issues

Organizational issues

- Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
- Is it standard organizational practice to develop a detailed **system specification**?
- Will **customer** representatives be available to **provide feedback** of system increments?
- Can informal agile development fit into the **organizational culture** of detailed documentation?

Chapter 2 Agile Software Development 78

이슈 ③ 조직

- 전통 공학 조직은 **계획 기반 개발 문화** (공학의 표준이니까)
- 상세 시스템 명세를 만드는 게 조직의 표준 관행인가?
- 고객 대표가 증분에 대한 피드백을 줄 수 있나?
- 비공식적 애자일이 **상세 문서 중심 조직 문화**에 맞을 수 있나?

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Large system development

Large system development

- Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed.
- Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.
- Large systems usually have a diverse set of stakeholders. It is practically impossible to involve all of these different stakeholders in the development process.

대규모 시스템 개발

- 대형 시스템과 개발 프로세스는 **외부 규칙·규제**에 제약받는 경우가 많다
- **조달·개발 기간이 길다** — 그동안 사람이 떠나 일관된 팀을 유지하기 어렵다
- **이해관계자가 다양**하다 — 모두를 개발에 참여시키는 건 사실상 불가능

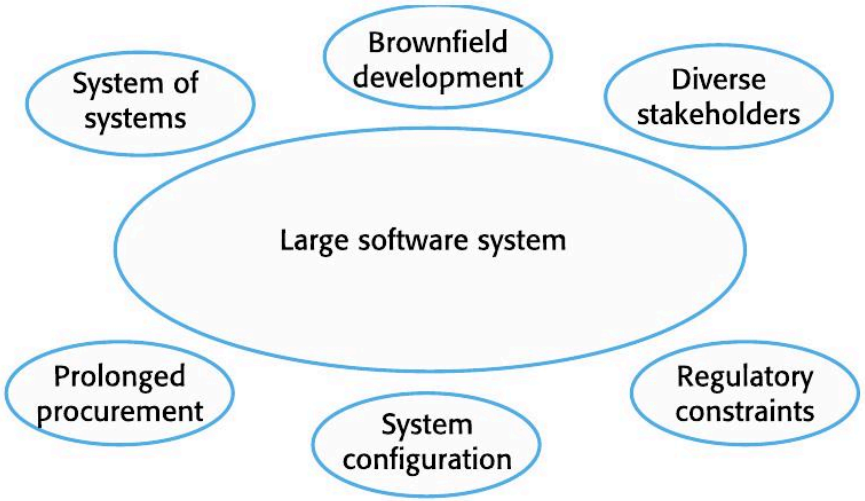
필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Large system development

[Factors in large systems]



대형 시스템의 요인 (그림)

- 대형 SW 시스템을 둘러싼 6요인
  - **System of systems (시스템의 시스템)**
  - **Brownfield development** (기존 시스템과 얽힌 개발)
  - **Diverse stakeholders (다양한 이해관계자)**
  - **Prolonged procurement (긴 조달)**
  - **System configuration (시스템 구성)**
  - **Regulatory constraints (규제 제약)**

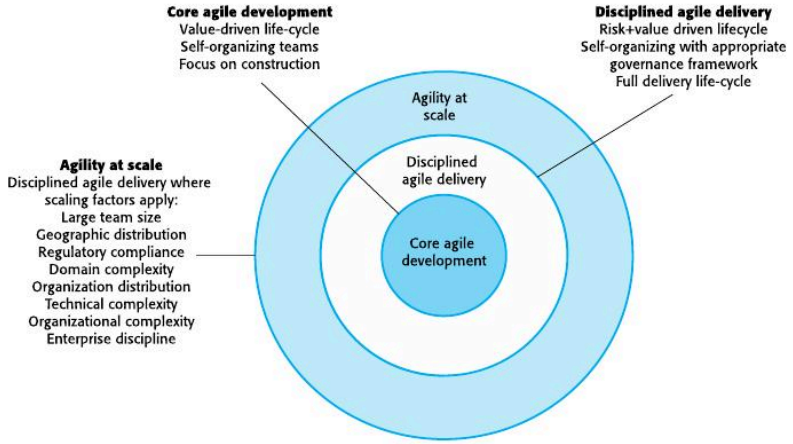
필기 ✍️

★ 시험 6요인 — 그림 빈칸 문제 대비

Scaling agile methods

Large system development

[Agile for Large system: IBM's agility at scale model]



Chapter 2 Agile Software Development 81

대규모 애자일: IBM의 agility at scale 모델 (그림)

- 동심원 3겹
  - 안쪽 **Core agile development** — 가치 주도 생명주기, 자기 조직화 팀, 구현에 집중
  - 가운데 **Disciplined agile delivery** — 위험+가치 주도 생명주기, 적절한 거버넌스 안의 자기 조직화, 전체 전달 생명주기
  - 바깥 **Agility at scale** — 규모 요인이 적용되는 disciplined agile delivery
    - 큰 팀, 지리적 분산, 규제 준수, 도메인 복잡도, 조직 분산, 기술 복잡도, 조직 복잡도, 기업 규율

필기 ✍️

— 넘어가도 되는 쪽

Scaling agile methods

Large system development

[Agile for large system: Multi-team Scrum]

Role replication	Each team has a Product Owner for their work component and ScrumMaster.
Product architects	Each team chooses a product architect and these architects collaborate to design and evolve the overall system architecture.
Release alignment	The dates of product releases from each team are aligned so that a demonstrable and complete system is produced.
Scrum of Scrums	There is a daily Scrum of Scrums where representatives from each team meet to discuss progressand plan work to be done.

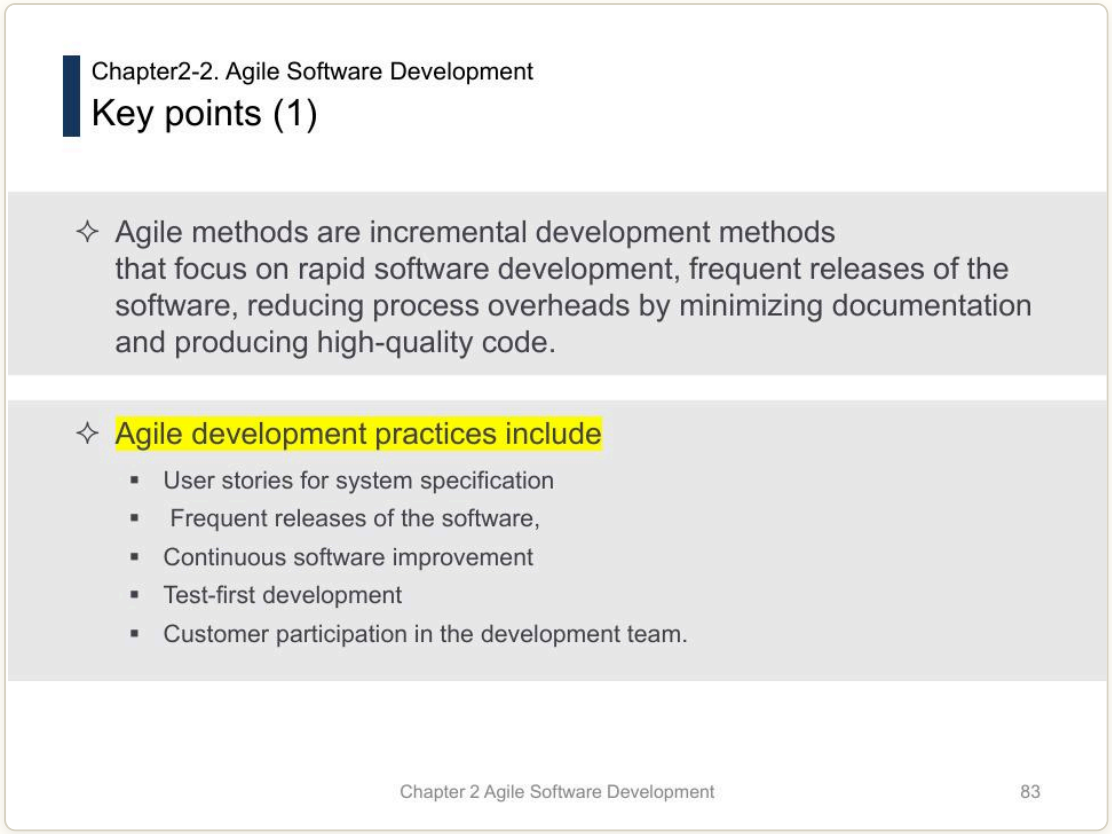
Chapter 2 Agile Software Development 82

대규모 애자일: 멀티팀 스크럼

- **Role replication (역할 복제)** — 팀마다 자기 작업 컴포넌트의 **제품 소유자와 스크럼 마스터**
- **Product architects (제품 아키텍트)** — 팀마다 아키텍트를 뽑고, 이들이 협력해 전체 아키텍처를 설계·진화
- **Release alignment (릴리스 정렬)** — 팀들의 릴리스 날짜를 맞춰 시연 가능한 완전한 시스템을 만듦
- **Scrum of Scrums (스크럼의 스크럼)** — 팀 대표들이 **매일** 만나 진척을 논의하고 할 일을 계획

필기 ✍️

★ 시험 멀티팀 스크럼 4요소: 역할 복제 · 제품 아키텍트 · 릴리스 정렬 · Scrum of Scrums



핵심 정리 ①

- 애자일 = 빠른 개발, 잦은 릴리스, 문서 최소화로 프로세스 오버헤드 감소, 고품질 코드에 집중하는 **점증적 개발 방법**
- 애자일 개발 실천
  - 시스템 명세용 사용자 스토리 / 잦은 릴리스
  - 지속적 SW 개선(리팩토링) / 테스트 우선 개발
  - 개발팀에 고객 참여

필기 ✍️

— 넘어가도 되는 쪽

Chapter2-2. Agile Software Development

Key points (2)

✧ Scrum is an agile method that provides a project management framework.
- It is centred round a set of sprints, which are fixed time periods when a system increment is developed.

✧ Many practical development methods are a mixture of plan-based and agile development.

✧ Scaling agile methods for large systems is difficult.
- Large systems need up-front design and some documentation and organizational practice may conflict with the informality of agile approaches.

Chapter 2 Agile Software Development 84

핵심 정리 ②

- 스크럼은 **프로젝트 관리 프레임워크**를 주는 애자일 방법
  - 시스템 증분을 개발하는 **고정 기간인 스프린트** 중심
- 실제 개발 방법은 대부분 **계획 기반 + 애자일 혼합**
- 대형 시스템에 애자일 규모 확장은 **어렵다**
  - 대형 시스템은 **사전 설계와 어느 정도 문서**가 필요하고, 조직 관행이 애자일의 비공식성과 충돌할 수 있다

필기 ✍️

— 넘어가도 되는 쪽